

Índice

1. Introducción	4
2. Escenarios	4
3. Vista Lógica	5
3.1. Grafo Acíclico Dirigido (DAG)	5
3.1.1. Pipeline de filtrado	5
3.1.2. Q1 — Cuentas con transacciones en USD menores a 50	5
3.1.3. Q2 — Máxima transacción en USD por banco	6
3.1.4. Q3 — Transacciones por debajo de la centésima parte del promedio	6
3.1.5. Q4 — Pares de cuentas con cinco intermediarios	7
3.1.6. Q5 — Conteo de microtransacciones Wire/ACH	7
4. Vista de Procesos	8
4.1. Diagrama de Secuencia	8
4.2. Diagramas de Actividad	9
5. Manejo de EOF	19
5.1. Topología de anillo	19
5.2. Garantía de entrega confiable	19
5.3. Trade-offs	20
5.4. Consideraciones frente a la tolerancia a fallos	20
6. Vista Física y de Despliegue	20
6.1. Tipos de nodos lógicos	20
6.1.1. Actores	20
6.1.2. Workers	21
6.2. Diagrama de Robustez	22
6.3. Diagrama de Despliegue	29
7. Vista de Desarrollo	30
7.1. Diagrama de Paquetes	30
8. Tolerancia a fallas	31
8.1. Inyección de fallas: Chaos Monkey	32
8.2. Detección y recuperación: cluster de centinelas	32
8.2.1. Detección por heartbeat	32
8.2.2. Recuperación	32
8.2.3. Elección de líder: algoritmo Bully	33
8.2.4. Recuperación de los propios centinelas	33
8.3. Topología de comunicación	34
8.3.1. Trade-offs	35

8.4. Reconexión del cliente	36
8.5. Persistencia y recuperación del estado	36
8.5.1. ¿Cuándo se persiste?	36
8.5.2. ¿Cómo se persiste?	37
8.5.3. Deduplicación	38
9. Planificación y Organización	39
9.1. Listado de Tareas a Ejecutar	39
9.2. División del Trabajo entre Integrantes	40

1. Introducción

El presente documento detalla la arquitectura y el diseño de un sistema distribuido cuyo propósito es analizar extractos de transacciones entre cuentas bancarias para detectar anomalías. Este análisis se enmarca en la detección del lavado de activos, un delito que consiste en cambiar bienes o dinero obtenidos mediante actos ilícitos por dinero legítimo. Para lograr esto, los infractores utilizan la red digital de pagos aplicando diversas estrategias de ofuscación, tales como transacciones Fan-out, Fan-in, patrones Scatter-Gather, grafos Bipartitos y Stack.

Para hacer frente a esta problemática, se diseñó una solución optimizada para entornos multi-computadoras. La arquitectura propuesta permite escalar los elementos de cómputo horizontalmente para soportar el incremento en los volúmenes de información y clientes simultáneos a procesar. A nivel de infraestructura, el sistema se apoya en un Middleware que abstrae la comunicación basada en grupos. Asimismo, el flujo de procesamiento soporta una única ejecución continua y provee mecanismos de cierre ordenado.

Los datos utilizados para el procesamiento provienen del dataset de simulaciones bancarias de IBM (disponible en Kaggle.), integrándose además con la API Frankfurter para obtener las cotizaciones diarias necesarias para las conversiones de divisas a USD.

2. Escenarios

Query 1 — Micropagos (Fan-out / Fan-in): Se buscan todas las transacciones en dólares estadounidenses (USD) cuyo monto sea estrictamente menor a \$50. Para cada una se reportan la cuenta de origen, la cuenta de destino y el monto transferido. Montos pequeños y frecuentes son un indicador típico de los patrones *fan-out* y *fan-in*, donde una cuenta dispersa o concentra fondos en pequeñas fracciones para evitar los umbrales de reporte automático de las entidades bancarias.

Query 2 — Máximos por Entidad Bancaria: Para cada banco de origen presente en el conjunto de transacciones en USD, se identifica la transferencia de mayor monto. El resultado incluye el nombre del banco, la cuenta emisora y el monto correspondiente. Esta consulta permite detectar cuentas que concentran movimientos de alto valor dentro de una misma entidad, lo cual puede ser indicativo de operaciones atípicas o de una cuenta central en un esquema de lavado.

Query 3 — Anomalías Históricas (Stack): Se analiza el período [2022-09-06, 2022-09-15] comparándolo con el período histórico de referencia [2022-09-01, 2022-09-05]. Para cada formato de pago, se calcula el promedio de montos en USD del período de referencia. Luego se reportan las transacciones del período analizado cuyo monto sea inferior a un centésimo (1 %) de dicho promedio. Este tipo de anomalía corresponde al patrón *stack*: transferencias acumuladas de montos ínfimos a lo largo del tiempo, diseñadas para pasar desapercibidas.

Query 4 — Patrón Scatter-Gather: Se identifican las cuentas que participan en un esquema de *scatter-gather* con exactamente una cuenta intermediaria. La condición es que la cuenta de origen haya realizado transferencias en USD hacia al menos 5 cuentas intermediarias distintas durante el período [2022-09-01, 2022-09-05], y que cada una de esas cuentas intermediarias haya transferido fondos hacia una misma cuenta de destino final. Este patrón describe el flujo *fan-out* desde un origen, seguido de un *fan-in* hacia un destino, utilizando cuentas intermediarias.

Query 5 — Micropagos Internacionales (Wire / ACH): Se contabilizan las transacciones del período [2022-09-01, 2022-09-05] realizadas mediante los formatos de pago *Wire* o *ACH* cuyo monto, una vez convertido a USD utilizando el tipo de cambio diario provisto por la API de Frankfurter, sea estrictamente menor a \$1. La conversión dinámica permite detectar micropagos internacionales que, expresados en moneda local, pueden no parecer sospechosos pero que en USD revelan un volumen anómalo de operaciones de bajísimo valor.

3. Vista Lógica

En esta sección presentamos la vista lógica del sistema, cuyo objetivo es describir el flujo de datos desde la fuente de datos enviado por el cliente hasta el sumidero final con los resultados consolidados que se devuelven al mismo cliente. El énfasis está puesto en las transformaciones que sufre cada tupla, sin todavía comprometernos con cómo se distribuyen físicamente esas transformaciones entre nodos.

3.1. Grafo Acíclico Dirigido (DAG)

Representamos el flujo lógico de los datos mediante un Grafo Acíclico Dirigido (DAG). Cada nodo es una transformación lógica (**source**, **router**, **map**, **filter**, **reducer**, **aggregator**, **joiner** o **sink**) y cada arista lleva la tupla concreta que se transmite entre etapas a través de un *exchange* o *queue* de RabbitMQ.

Este esquema de parciales y combinación es posible porque las agregaciones involucradas (máximos, sumas, conteos) dan el mismo resultado tanto si se computan de una sola vez como por partes. Cuando la operación no es directamente combinable (como el promedio de Q3, dado que el promedio de promedios no equivale al promedio real) se acumulan en su lugar magnitudes que sí lo son (**suma** y **conteo**) y la operación final se aplica una única vez sobre el agregado.

Para evitar un único diagrama sobrecargado, descomponemos el grafo en un **pipeline de filtrado**, común a todas las consultas, y **cinco sub-DAG independientes**, uno por consulta. Todos los sub-DAG parten de las *transacciones filtradas* que produce el pipeline inicial y desembocan en sus respectivos resultados.

3.1.1. Pipeline de filtrado

La fuente **Transacciones** produce una tupla por cada transacción con todos sus campos (**TxID**, **FromBank**, **FromAcc**, **ToBank**, **ToAcc**, **PaymentCurrency**, **Amount**, **PaymentFormat**, **Date**). Esa tupla atraviesa una cadena de filtros que actúa como *smart router*: evalúa predicados sobre moneda, período, monto y medio de pago, y encamina cada transacción hacia las colas terminales que alimentan cada consulta. Los predicados sobre el monto no forman parte de esta cadena común, sino que quedan a cargo de cada consulta que los necesita. La notación **TRANSACCIÓN*** denota la tupla ya filtrada y proyectada que las ramas Q1–Q5 reciben como entrada.

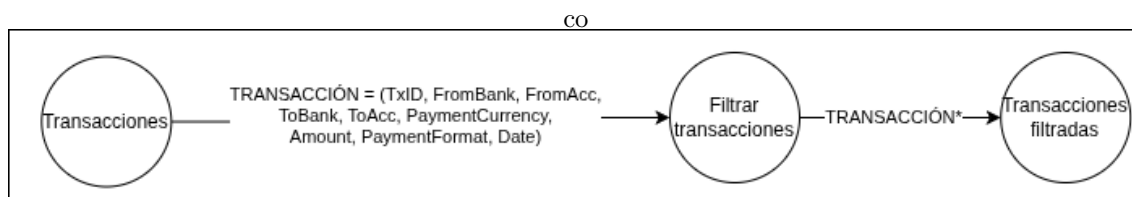


Figura 1: Pipeline base de filtrado, común a todas las consultas.

3.1.2. Q1 — Cuentas con transacciones en USD menores a 50

Es la rama más simple, donde cada tupla se procesa de forma independiente. Recibe las transacciones en USD, conserva únicamente aquellas cuyo monto es inferior a 50, proyecta cada una a (**FromAcc**, **ToAcc**, **Amount**) y la emite directamente al resultado.

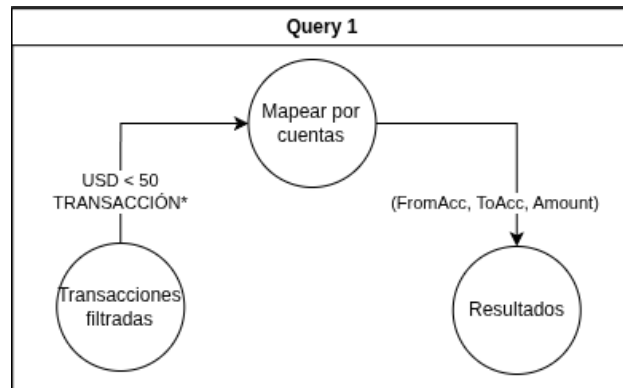


Figura 2: Sub-DAG de la consulta Q1.

3.1.3. Q2 — Máxima transacción en USD por banco

El máximo se calcula en dos etapas. Las transacciones en USD se reparten por banco de origen y, dentro de cada partición, se obtiene la máxima transacción, produciendo un máximo parcial (*FromBank*, *FromAcc*, *MaxAmount*). Esos parciales se combinan luego en el máximo global de cada banco. En paralelo, el cliente envía el catálogo de bancos (*BankID*, *BankName*), que se cruza con cada máximo para incorporar el nombre del banco, produciendo (*BankID*, *BankName*, *FromBank*, *FromAcc*, *MaxAmount*).

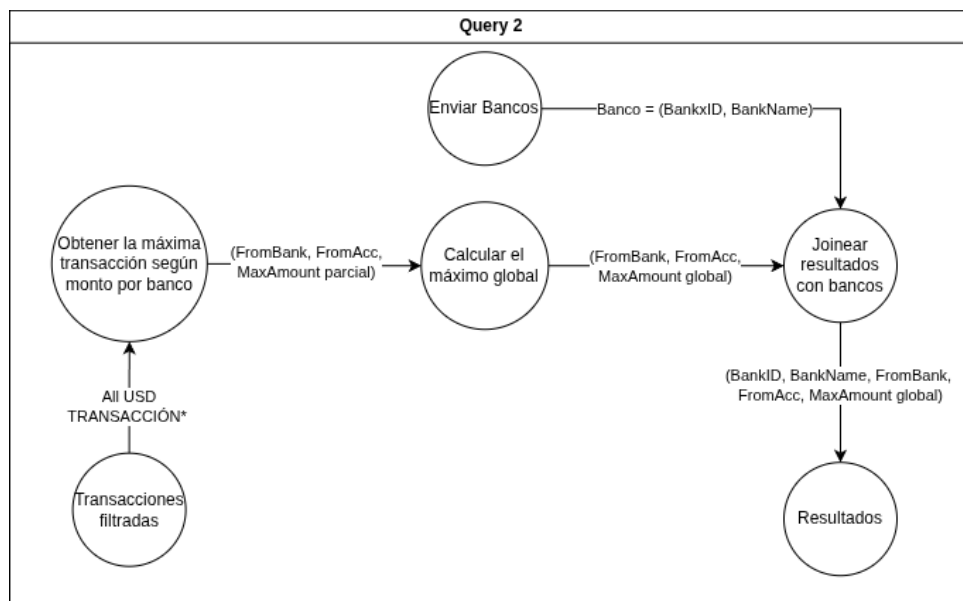


Figura 3: Sub-DAG de la consulta Q2.

3.1.4. Q3 — Transacciones por debajo de la centésima parte del promedio

Q3 ejecuta dos sub-ramas en paralelo. Sobre el período de referencia [2022-09-01, 2022-09-05], las transacciones se agrupan por *PaymentFormat* para acumular (*suma*, *conteo*) parciales que luego se combinan, obteniendo el promedio de cada medio de pago. La otra sub-rama retiene las transacciones del período analizado [2022-09-06, 2022-09-15] a la espera de ese promedio. Una vez disponible, se conservan únicamente las transacciones cuyo monto es inferior a la centésima parte del promedio de su *PaymentFormat*, emitiendo (*FromAcc*, *PaymentFormat*, *Amount*).

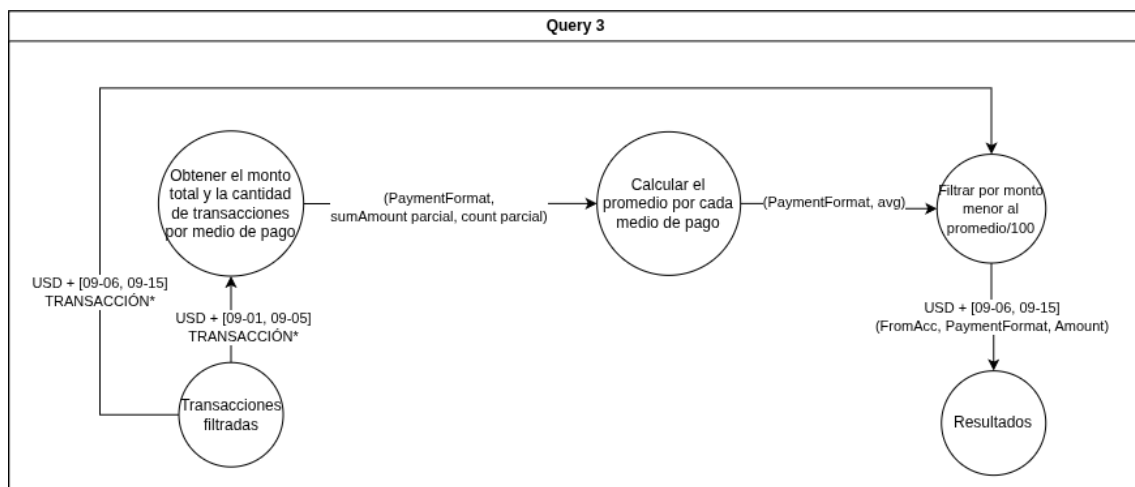


Figura 4: Sub-DAG de la consulta Q3.

3.1.5. Q4 — Pares de cuentas con cinco intermediarios

Para detectar cadenas de intermediarios, cada transacción en USD del período [2022-09-01, 2022-09-05] se duplica y se reparte por su cuenta de origen y por su cuenta de destino (Acc, OtherAcc, ShardingAcc), agrupando las que comparten una cuenta. Antes de la etapa más costosa se aplica un **pre-filtrado**, donde se conservan únicamente las cuentas con cinco o más contrapartes distintas —es decir, con al menos cinco puntos de entrada o de salida— y se descarta el resto, que no puede formar parte del patrón. Esto reduce drásticamente el volumen que llega a la reconstrucción de caminos. Sobre las transacciones que sobreviven se arman los caminos de tres cuentas (FromAcc, IntermediateAcc, ToAcc) y, combinando los parciales, se retienen los pares (origen, destino) conectados a través de al menos cinco cuentas intermedias distintas, emitiendo (FromAcc, ToAcc).

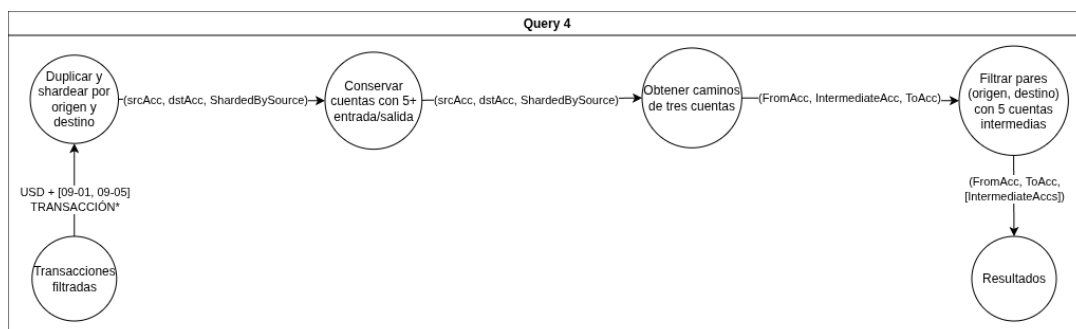


Figura 5: Sub-DAG de la consulta Q4.

3.1.6. Q5 — Conteo de microtransacciones Wire/ACH

Sobre las transacciones de tipo Wire/ACH del período [2022-09-01, 2022-09-05], se convierte el monto a USD y se descartan las conversiones iguales o superiores a 1 USD. El conteo se realiza en dos etapas, donde cada partición cuenta localmente sus transacciones (**partial_count**) y esos conteos parciales se suman en el total global (**total_count**).

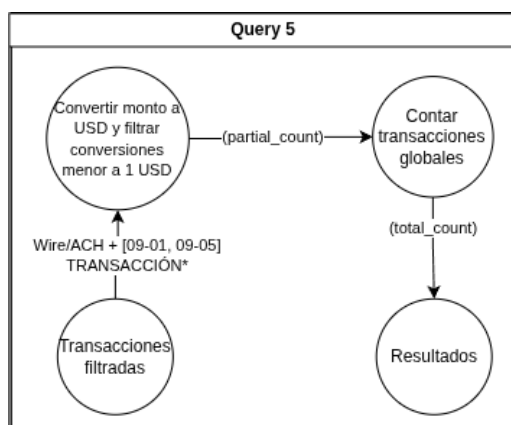


Figura 6: Sub-DAG de la consulta Q5.

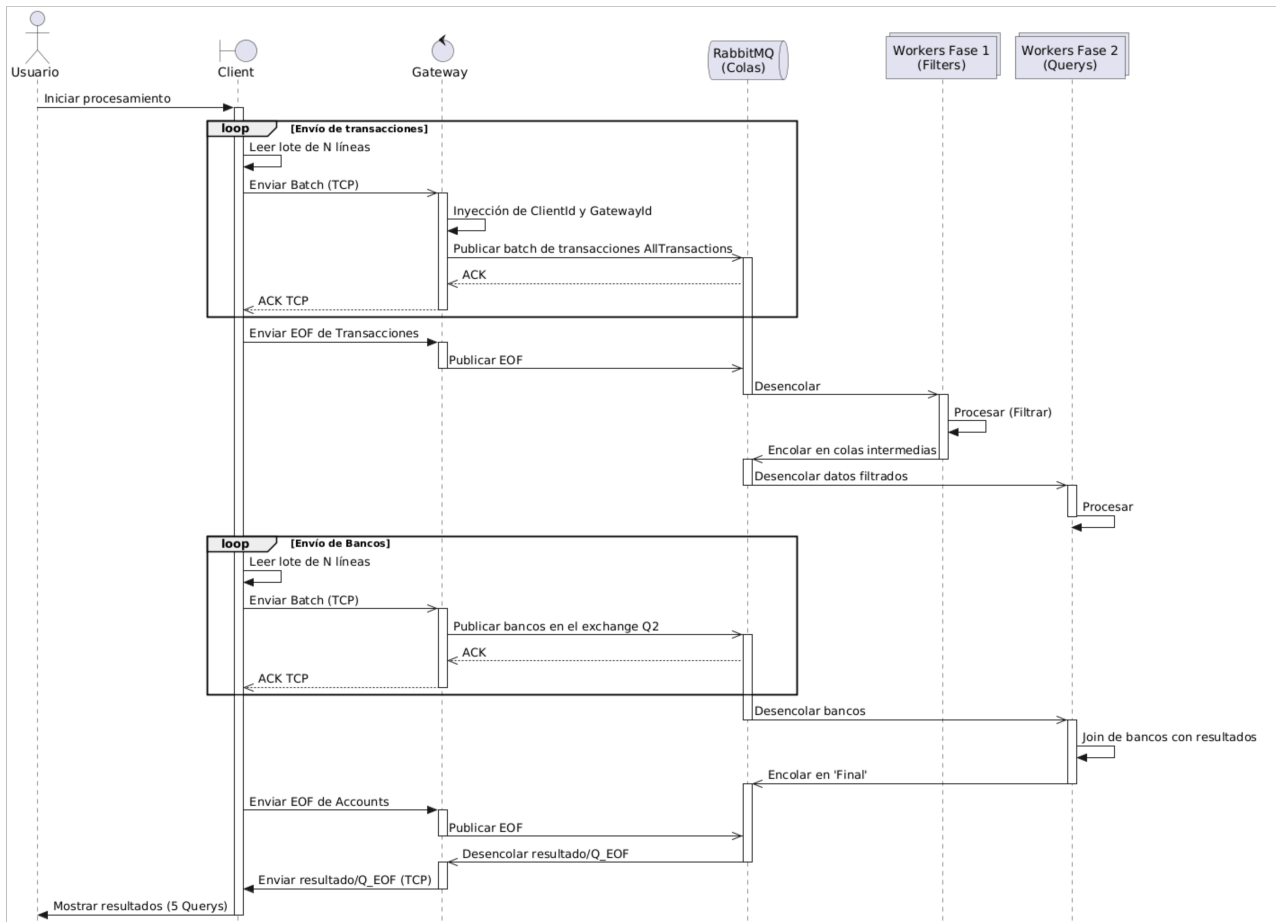
Todas las ramas convergen en el *Final Joiner*, que envía el resultado parcial de cada cliente por cada consulta hacia el Gateway. Este la reenvía al cliente por el mismo socket TCP por el que llegaron las transacciones.

4. Vista de Procesos

En la presente sección describimos la dinámica del sistema, es decir, cómo interactúan los distintos componentes a lo largo de la ejecución y cómo se coordinan para producir los resultados de cada consulta. Presentamos en primer lugar un diagrama de secuencia que ilustra las interacciones entre las entidades principales, y a continuación los diagramas de actividad que detallan el comportamiento interno de cada componente.

4.1. Diagrama de Secuencia

El diagrama que sigue resume el flujo completo de una sesión de cliente: el establecimiento de la conexión TCP, el envío de *batches* de transacciones y bancos, la propagación de éstas hacia las distintas *Queues*, el procesamiento por las consultas Q1–Q5, la convergencia en el *Final Joiner* y la entrega final de resultados al cliente.

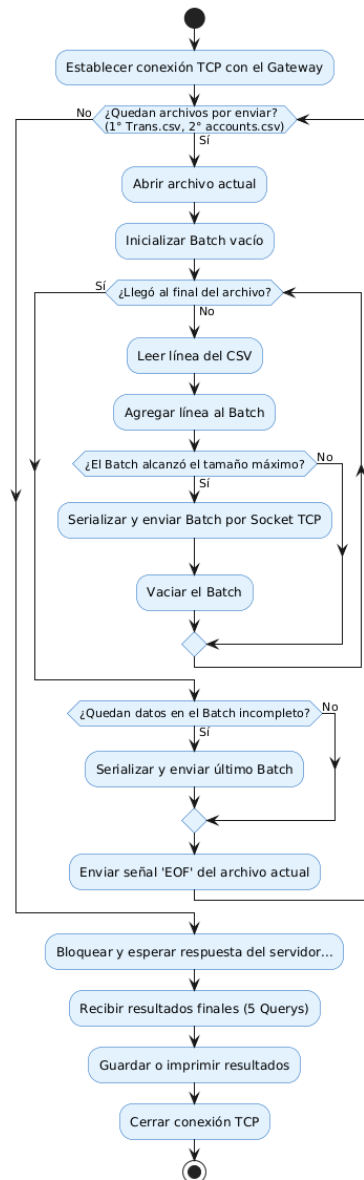


Resulta importante destacar que la comunicación entre cliente y *gateway* es síncrona y orientada a la conexión (TCP/IP), mientras que toda la comunicación interna entre el *gateway* y los *workers* es asíncrona y mediada por RabbitMQ. Esta separación permite desacoplar la velocidad de ingreso de los datos del cliente de la velocidad de procesamiento del *backend*, y aislar al cliente de los detalles de coordinación interna del sistema.

4.2. Diagramas de Actividad

A continuación se presentan los diagramas de actividad de los distintos componentes del sistema. Cada uno describe el procesamiento que un *worker* realiza por cada mensaje recibido, así como su comportamiento al detectar la señal de fin de procesamiento (EOF). El protocolo concreto mediante el cual se garantiza que el EOF refleja el cierre de los datos del cliente se describe en la sección *Manejo de EOF*; en lo que sigue asumimos que esa garantía existe.

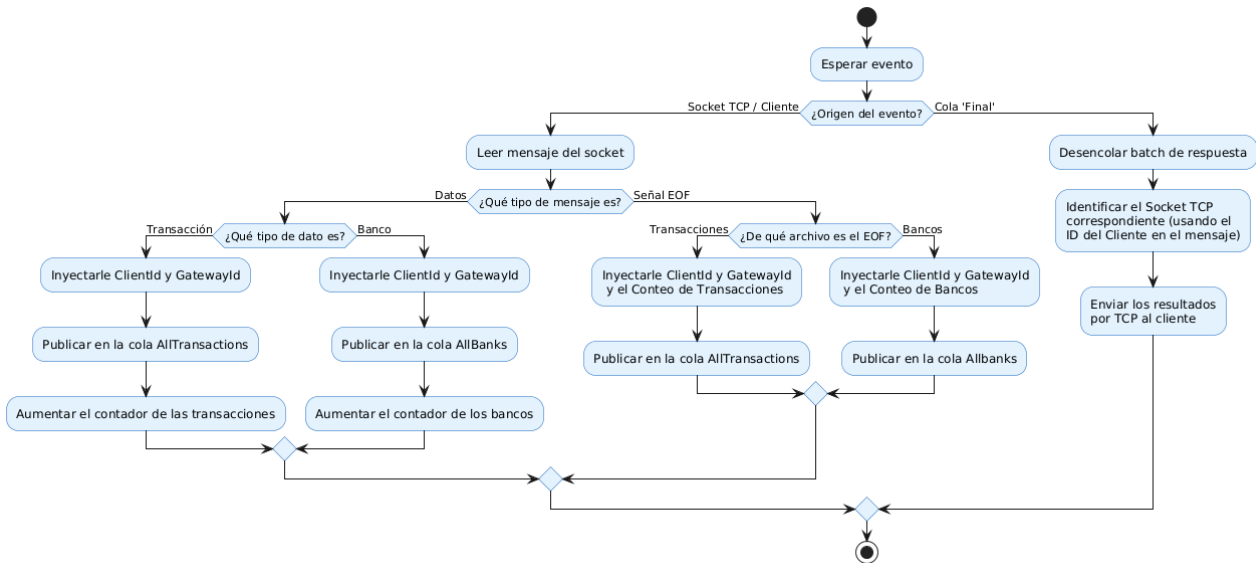
- **Cliente.** El cliente selecciona aleatoriamente uno de los *gateways* disponibles e intenta establecer una conexión TCP. En caso de fallo, reintenta con backoff exponencial, rotando entre los *gateways* disponibles hasta establecer la conexión. Una vez conectado, lee el archivo *Trans.csv* línea por línea y arma *batches* de tamaño configurable que envía por el socket. Al alcanzar el final del archivo serializa el último *batch* y emite la señal EOF sobre el mismo socket. Una vez hecho esto, sigue exactamente los mismos pasos para el archivo *accounts.csv*. A continuación se bloquea a la espera de los resultados que son recibidos y guardados en disco progresivamente. Al llegar los 5 EOF (1 por query) el cliente se cierra.



- **Gateway.** El gateway actúa como proxy entre los clientes TCP y RabbitMQ. Al recibir una nueva conexión, realiza un *handshake* con el cliente asignándole un **ClientID** UUID único. A partir de ese momento maneja dos flujos en paralelo por cliente.

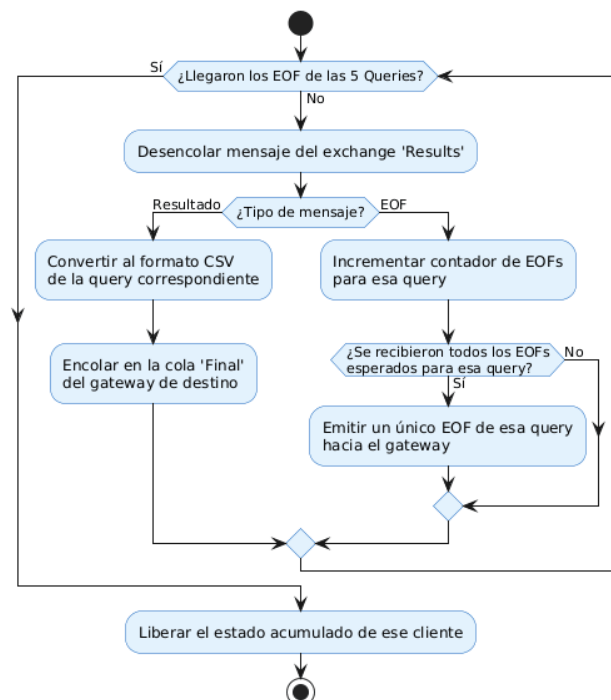
En el flujo de **entrada**, por cada *batch* de transacciones o cuentas recibido por TCP lo etiqueta con el **ClientID** y **GatewayID** correspondiente y lo publica en la cola transacciones o cuentas respectivamente, confirmando la recepción al cliente con un **ACK**. Lo mismo ocurre con los mensajes EOF de cada archivo.

Además en el flujo de **salida**, se consume continuamente de la cola **final_{gatewayID}**. Cada resultado llega con **ClientID** y **GatewayID** embebidos; el gateway identifica el cliente destinatario en su registro y le reenvía los resultados por TCP. Utiliza un esquema *at-least-once*: el ACK de AMQP se emite recién cuando el cliente confirma haber recibido el *batch*. La sesión se cierra al entregar los 5 EOF de queries esperados.



- **Final Joiner.** Consume del *exchange results* a través de su propia cola dedicada (shar-deado: cada réplica recibe una fracción de los mensajes). Para cada mensaje entrante realiza dos tareas: convierte el formato binario interno al formato CSV de salida correspondiente a cada consulta (Q1–Q5), y lo encola en la cola *final_{gatewayID}* del gateway de origen, preservando el *ClientID* para que el gateway pueda rutear la respuesta al cliente correcto.

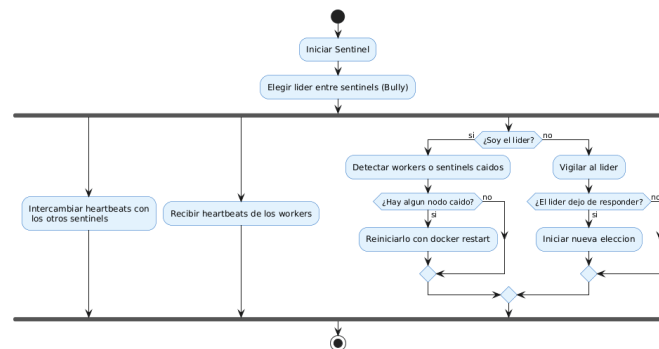
En cuanto al tratamiento de los EOF, cada consulta configurada mantiene un contador de los EOF que debe recibir. En particular, espera *EXPECTED EOFs_Q{n}* EOFs provenientes de upstream (emitidos por las réplicas terminales de cada rama) antes de propagar un único EOF downstream hacia el gateway. De esta manera, se abstrae la multiplicidad interna de réplicas y se preserva la interfaz esperada por el gateway y, en consecuencia, por el cliente al enviar un solo EOF por consulta.



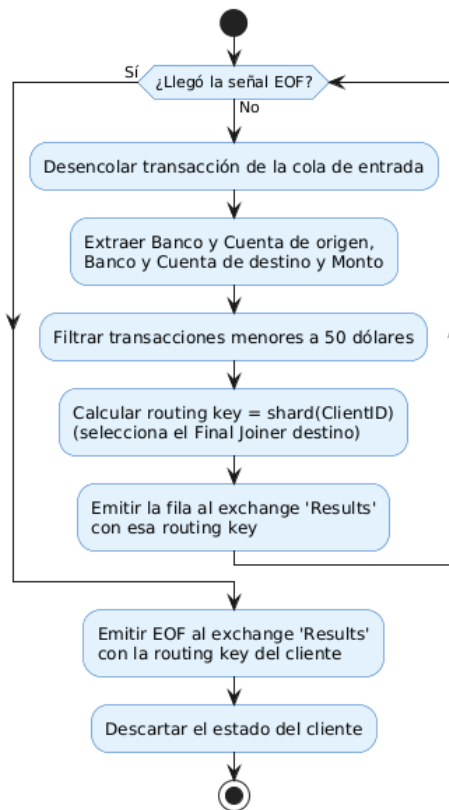
- **Sentinel.** Los *sentinels* conforman el subsistema de tolerancia a fallas: detectan nodos caídos y los reviven. Al iniciar, cada réplica levanta tres transportes (UDP para *heartbeats* entre sentinels, TCP para los mensajes de control del algoritmo de elección y UDP para los *heartbeats* de los workers) y participa en una elección de líder mediante el algoritmo *Bully*, en la que gana el sentinel de mayor ID.

Una vez elegido el líder, cada sentinel ejecuta varias tareas en paralelo. De forma continua intercambia *heartbeats* con sus pares y recibe los *heartbeats* que emiten los workers, registrando el último instante en que vio con vida a cada nodo. En paralelo, los sentinels que *no* son líder vigilan al líder: si su *heartbeat* no llega dentro de **LeaderTimeout**, disparan una nueva elección.

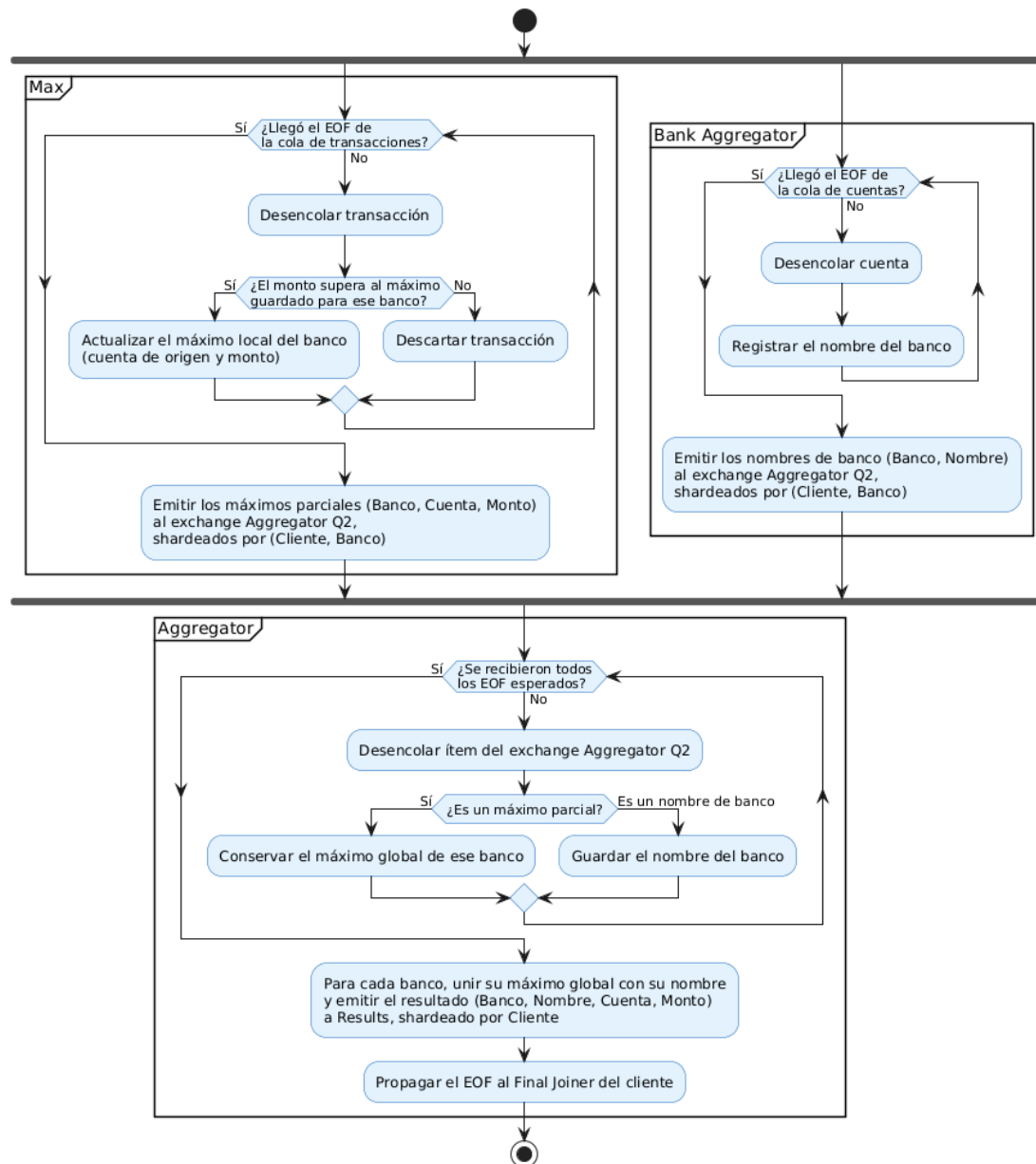
Este tipo de nodo está explicado más a detalle en la sección 8 Tolerancia a Fallas.



- **Query 1 (Micropagos).** Esta rama tiene una sola etapa (el *Filtro*), replicable en varias instancias, que opera de manera *online*. Por cada transacción evalúa si su monto en USD es inferior a 50. Las que no alcanzan ese umbral se descartan. Del resto extrae los datos relevantes (banco y cuenta de origen, banco y cuenta de destino, y monto) y los emite al *exchange Results shardeados* por **ClientID**, de modo que todos los resultados de un mismo cliente converjan en el *Final Joiner* que le corresponde. Al recibir el EOF lo reenvía a **Results** con la misma *routing key* del cliente y descarta su estado parcial.

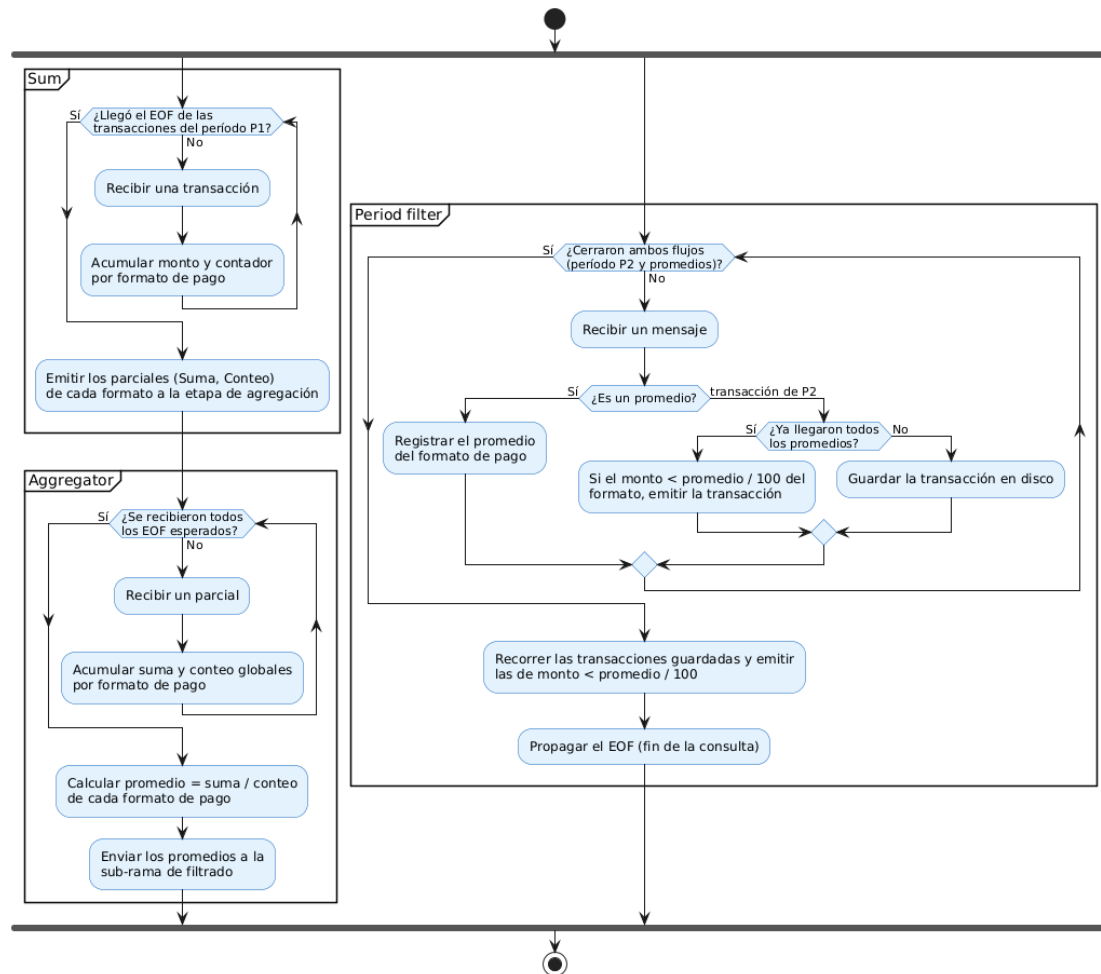


- **Query 2 (Máximos por Entidad Bancaria).** Se compone de tres etapas. La etapa *Max* mantiene un máximo local por banco mientras consume transacciones de su correspondiente cola, registrando la cuenta de origen y el monto de la mayor transacción de cada banco; al recibir el EOF emite sus máximos parciales (**Banco, Cuenta, Monto**) al *exchange Aggregator Q2*, *shardeados* por la clave (**Cliente, Banco**). En paralelo, la etapa *Bank Aggregator* consume el catálogo de cuentas y acumula el nombre de cada banco, que publica en el mismo *exchange* con idéntico *sharding*, de modo que el máximo de un banco y su nombre converjan en el mismo agregador. Finalmente, la etapa *Aggregator* consume ambos flujos: conserva el máximo global de cada banco y registra los nombres;. Cuando ha recibido todos los EOF esperados, junta cada máximo con el nombre de su banco y emite el resultado (**Banco, Nombre, Cuenta, Monto**) a *Results*, *shardeado* por cliente hacia su *Final Joiner*, junto con su EOF.

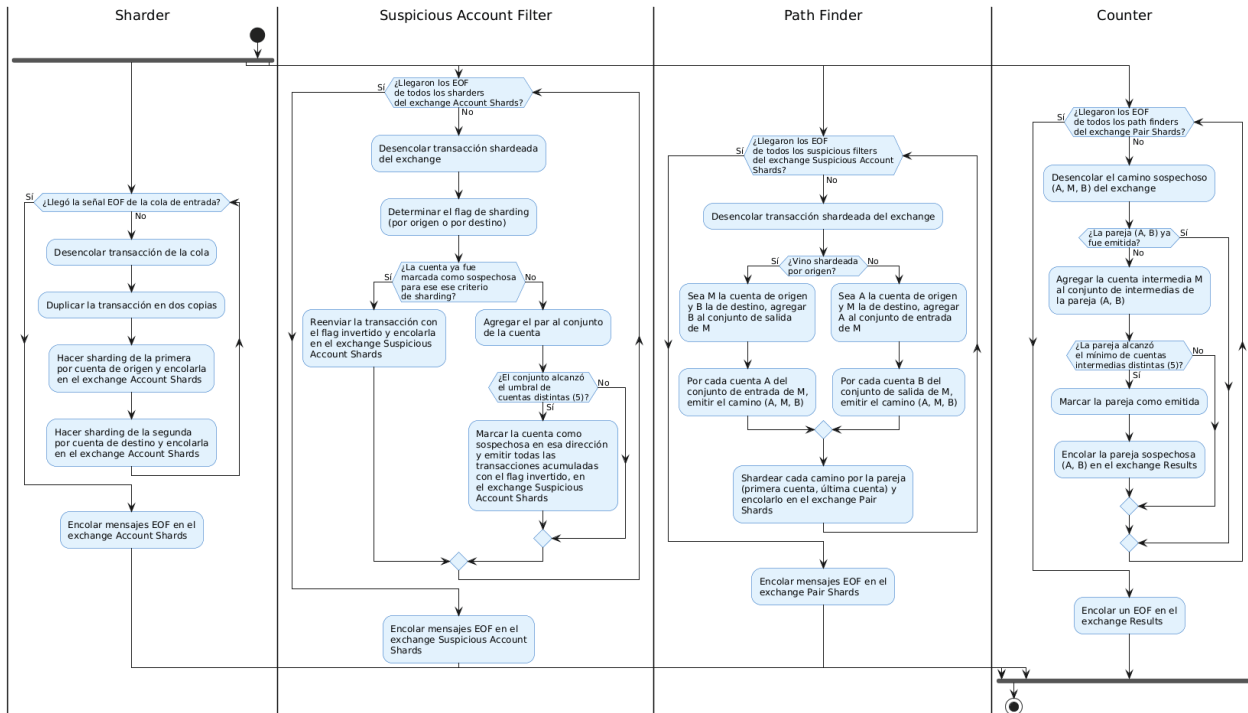


- **Query 3 (Anomalías Históricas).** Esta consulta se organiza en dos sub-ramas que operan concurrentemente. La primera sub-rama, encargada del cálculo de promedios, consta de dos etapas. En la etapa *Sum*, se procesan las transacciones correspondientes al período de referencia (P1), manteniendo para cada formato de pago una suma acumulada y un conteo parcial. Al recibir el EOF, estos valores parciales son enviados a la etapa de agregación. Luego, el *Aggregator* consolida los resultados recibidos, calcula el promedio asociado a cada formato de pago y lo distribuye hacia la sub-rama de filtrado.

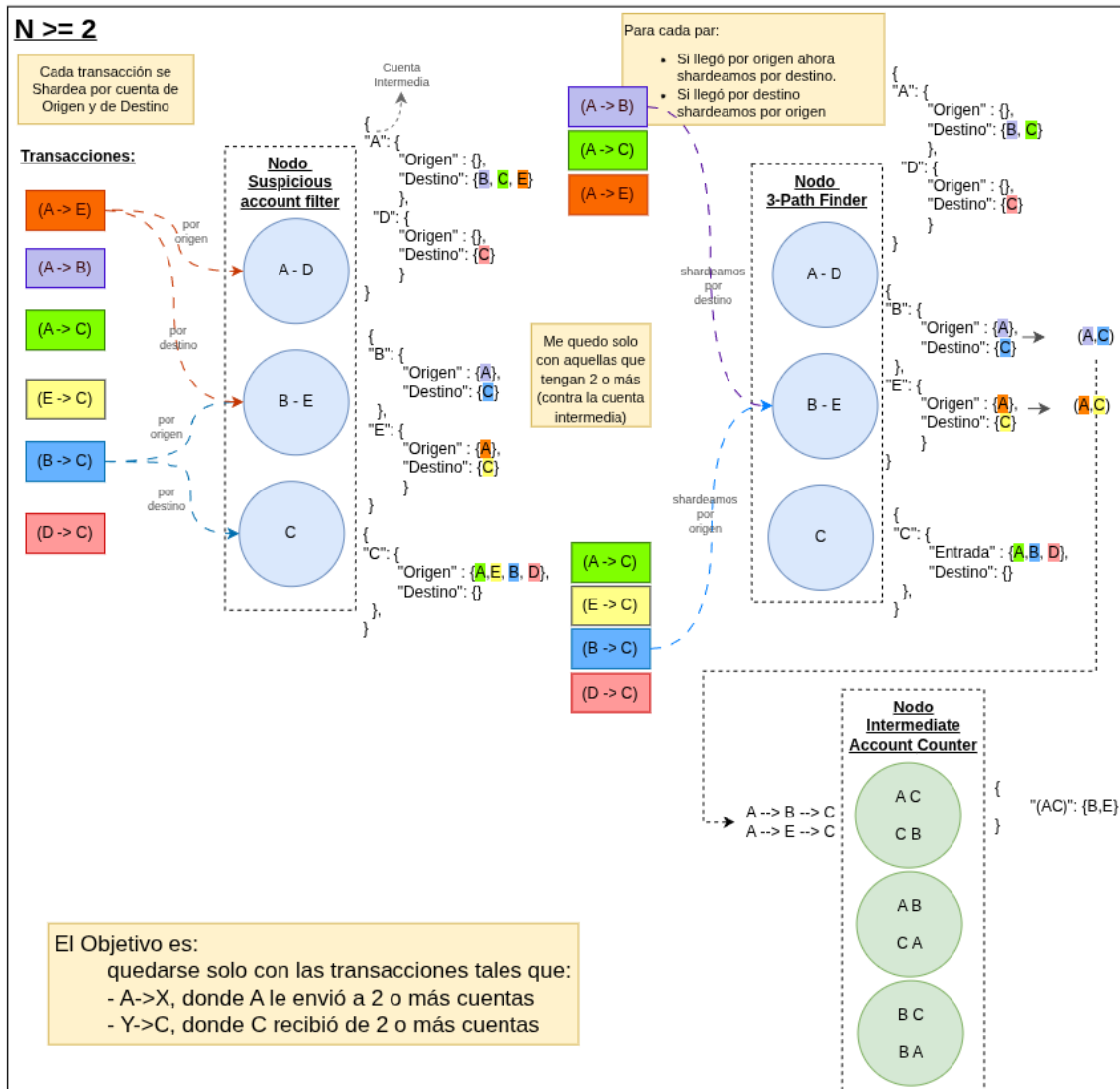
De manera simultánea, el *Period filter* consume dos flujos independientes: las transacciones del período analizado (P2) y los promedios calculados previamente. Mientras los promedios aún no están disponibles en su totalidad, las transacciones de P2 se persisten temporalmente en disco. Una vez completada la recepción de promedios, las nuevas transacciones se filtran a medida que arriban. Cuando ambos flujos finalizan, el componente recorre las transacciones almacenadas y emite como resultado únicamente aquellas cuyo monto es inferior a un centésimo del promedio correspondiente a su formato de pago. Finalmente, propaga el EOF asociado a la consulta.



- Query 4 (Scatter-Gather).** La rama está compuesta por cuatro etapas en *pipeline*. El *Sharder* recibe cada transacción y emite dos copias *shardeadas*: una marcada como 'shardeada por origen' y otra como 'shardeada por destino', ambas hacia el exchange *Account Shards*. El *Suspicious Account Filter* consume de dicho *exchange* y, por cada cuenta, acumula el conjunto de cuentas distintas con las que opera en cada dirección (entrada y salida); cuando una cuenta alcanza el umbral de cinco pares distintos en una dirección, la marca como sospechosa y emite hacia el exchange *Suspicious Account Shards* todas las transacciones acumuladas con el flag de *sharding* invertido (a partir de ese momento reenvía 1 a 1 las nuevas transacciones de esa cuenta). De esta forma solo avanzan las transacciones de cuentas con actividad suficiente. El *3-Path Finder* consume del exchange *Suspicious Account Shards* y, para cada cuenta intermedia, mantiene los conjuntos de cuentas que le envían fondos y a las que ella envía fondos; cada vez que descubre un nuevo camino sospechoso de la forma (origen, intermedia, destino), lo *shardea* por el par (origen, destino) y lo emite al exchange *Pair Shards*. Finalmente, el *Intermediate Account Counter* agrupa los caminos por par y emite a *Results* aquellos pares (origen, destino) que poseen al menos cinco cuentas intermedias distintas.



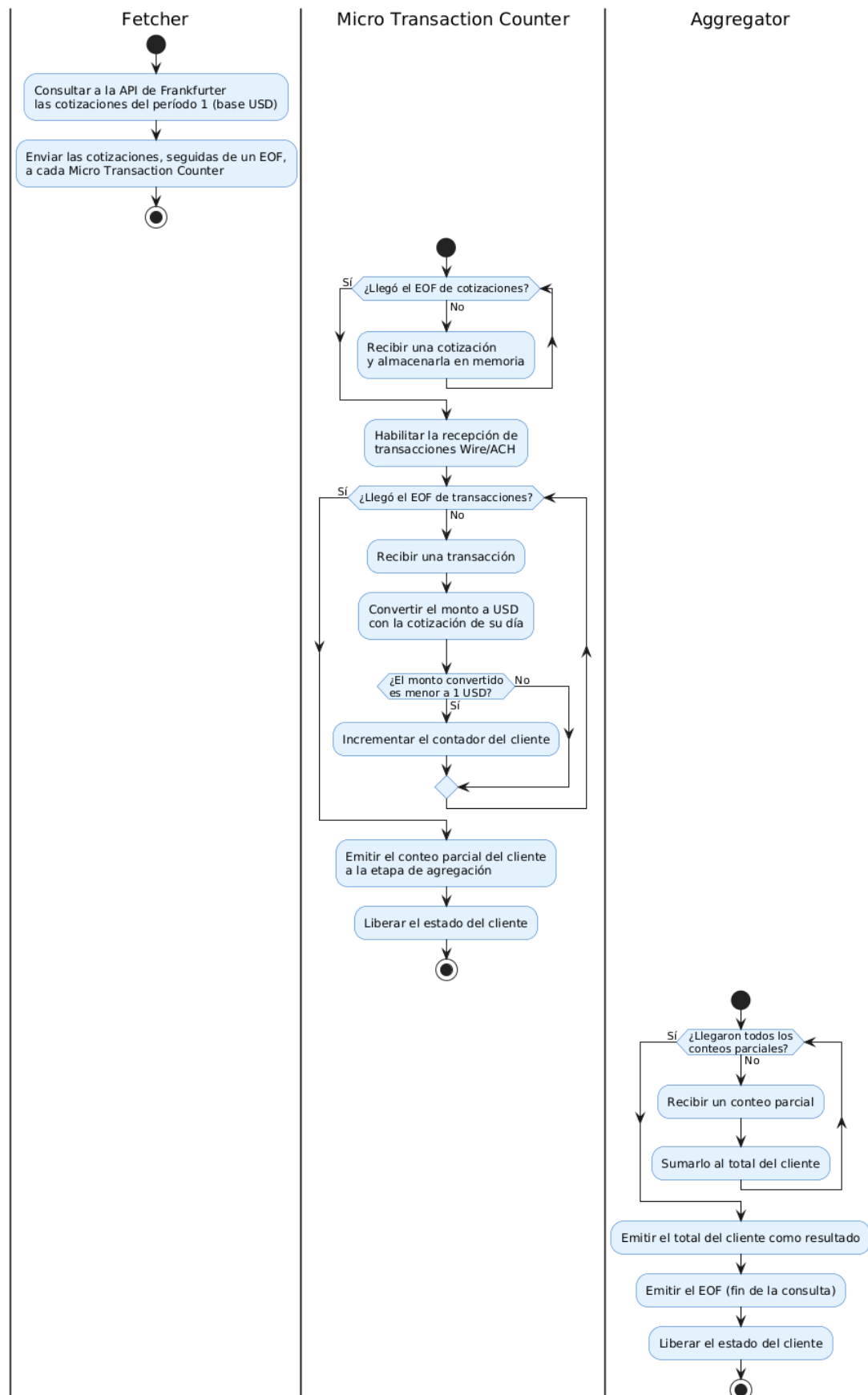
Para hacer tangible el algoritmo de Q4, el siguiente esquema muestra una ejecución de ejemplo. A la izquierda aparecen las transacciones de entrada, y en el centro la representación interna del *Suspicious Account Filter* y el *3-Path Finder* para cada cuenta intermedia: por cada cuenta se mantienen los conjuntos **Entrada** (cuentas que le envían fondos) y **Salida** (cuentas a las que envía fondos). En el *Suspicious Account Filter*, estos conjuntos se utilizan para filtrar solo a las cuentas con 5 o más cuentas a las que envió alguna transacción o aquellas que recibieron transacciones de 5 o más cuentas distintas. Por su parte, en el *3-Path Finder*, cada vez que la cuenta acumula un nuevo elemento en **Entrada** o en **Salida**, se emiten todos los nuevos caminos que se pueden armar con ese nuevo elemento. A la derecha se observa cómo el *Intermediate Account Counter* agrupa los caminos por el par (**origen, destino**) y mantiene el conjunto de cuentas intermediarias asociadas a cada par, emitiendo el par a **Results** en cuanto dicho conjunto alcanza las cinco cuentas intermedias distintas.



- **Query 5 (Micropagos Internacionales).** Esta consulta combina una etapa de inicialización de datos externos con dos etapas posteriores de procesamiento y agregación. En primer lugar, el componente *Fetcher* realiza una única consulta a la API de Frankfurter para obtener las cotizaciones correspondientes al período 1, utilizando USD como moneda base. Las cotizaciones recuperadas son distribuidas a todos los nodos *Micro Transaction Counter*, seguidas de un EOF que indica la finalización de la carga de referencias.

Cada *Micro Transaction Counter* almacena inicialmente las cotizaciones recibidas en memoria y recién comienza a procesar transacciones una vez recibido el EOF asociado a dicho flujo. A continuación, consume las transacciones de tipo Wire/ACH del período analizado y, para cada una, convierte el monto a USD utilizando la cotización correspondiente a la fecha de la operación. Si el valor convertido resulta inferior a 1 USD, incrementa un contador local asociado al cliente de la transacción.

Al finalizar el flujo de transacciones, cada nodo emite sus conteos parciales hacia la etapa de agregación. Finalmente, el *Aggregator* consolida los resultados provenientes de todos los *workers*, calcula el total de micropagos internacionales por cliente y publica el resultado final junto con el EOF de la consulta.



5. Manejo de EOF

Antes de avanzar con la vista física, resulta necesario describir el mecanismo mediante el cual el sistema garantiza el cierre ordenado del procesamiento de cada cliente. Dado que cualquier consulta involucra grupos de *workers* idénticos, donde cada réplica consume de su propia cola dedicada y procesa en paralelo una porción de los datos del cliente, la simple llegada de un mensaje de EOF a una de ellas no alcanza para asegurar que todos los datos del cliente hayan sido efectivamente procesados: aún pueden existir mensajes en tránsito, en *buffers* internos o en proceso de ser *ackeados*. Adoptar una política optimista en este punto comprometería la corrección de los resultados, lo cual es inadmisibles para una aplicación cuyo dominio es la detección de patrones en transacciones financieras.

5.1. Topología de anillo

Para resolver este problema se adoptó una topología de anillo entre los *workers* de cada grupo. Cuando mencionamos que un conjunto de N *workers* idénticos utiliza topología de anillo para el procesamiento de la señal EOF, cada *worker* i tiene una cola privada `worker_ringi` de la que consume y publica a la cola `worker_ring-(i+1) mód N` del siguiente. El EOF que publica el *gateway* viaja junto con los datos: de esa manera, llega a una réplica determinada del grupo, acompañado del total de filas que el *gateway* envió al respectivo tópico. Esa cifra es la *verdad de referencia* contra la cual el anillo decidirá si todos los datos fueron consumidos. continu

El protocolo se compone de dos rondas que viajan por el anillo:

- **Ronda de colecta.** El *worker* que recibió el EOF se declara *iniciador*, guarda el total esperado y arranca un mensaje con su propio conteo de filas procesadas para ese cliente. El mensaje recorre el anillo, y cada *worker* intermedio simplemente suma su conteo acumulado al valor que viene y reenvía al siguiente. Los *workers* intermedios no necesitan recordar el mensaje después de reenviarlo: su único aporte es agregar su parte a la suma parcial. Cuando el mensaje vuelve al iniciador, éste compara el total acumulado contra el total esperado. Si los números coinciden, dispara la ronda de envío. Si no coinciden, el iniciador repubblica el mismo EOF sobre la cola de la cual lo recibió, para que eventualmente algún *worker* (puede ser él mismo u otro) reinicie la colecta. La hipótesis subyacente es que, si la cuenta no dio, algún *worker* todavía tenía filas en *buffer* sin procesar, y darles una vuelta más les concede el tiempo necesario para drenar esos *buffers* y converger a la cuenta correcta.
- **Ronda de envío.** El iniciador *flushes* sus datos acumulados a la cola correspondiente junto con un EOF y envía un segundo mensaje por el anillo. Cada *worker* intermedio, al recibirlo, hace lo mismo: *flushes* sus datos acumulados con su propio EOF, borra el estado local de los datos que *flushes* y reenvía. Cuando el mensaje vuelve al iniciador, éste da por finalizada la ronda. El componente posterior espera N EOFs por cliente, los recibe gracias a la combinación de los *flushes* de cada *worker* y, recién entonces, procesa la consulta para ese cliente.

De esta manera, el sistema garantiza que ningún componente posterior procese datos de un cliente hasta que todos los *workers* hayan *flushado* sus aportes para ese cliente, lo cual asegura la consistencia y completitud del resultado.

5.2. Garantía de entrega confiable

La garantía de entrega confiable descansa sobre el hecho de que el EOF llega con el total de filas que el cliente envió (para ese tópico). Mientras ese total no coincida con la suma de lo que reportan los *workers*, el sistema se niega a hacer un flush de los datos parciales y repubblica el EOF. Cada reintento le concede a los *workers* “lentos” más tiempo para drenar sus *buffers*, y el sistema converge eventualmente al resultado correcto. Esto es especialmente importante para consultas que requieren procesar grandes volúmenes de datos, ya que permite manejar situaciones donde algunos *workers* resultan más lentos que otros sin comprometer la integridad del procesamiento.

5.3. Trade-offs

- **Mensajes del protocolo.** Siendo N la cantidad de *workers* en el anillo, este utiliza $2N$ mensajes por cliente en el caso optimista. La ronda de colecta consume N saltos para que el mensaje vuelva al iniciador, y la ronda de envío otros N saltos. El costo es lineal en la cantidad de *workers* del grupo.
- **Costo del reintento.** En presencia de transacciones en *buffer* cuando arranca la primera colecta, el conteo no cierra y se requiere una vuelta de N mensajes adicionales por reintento. Si un mensaje en vuelo se atrasa demasiado, pueden producirse varias vueltas de reintento, lo cual agrega latencia. Sin embargo, esa latencia es el precio de garantizar la entrega confiable sin perder datos, y resulta un compromiso razonable dado que el sistema converge eventualmente al resultado correcto.
- **Frente a un esquema centralizado.** Una alternativa considerada y descartada fue centralizar el conteo en un nodo coordinador mediante confirmaciones explícitas de cada *worker*. Esa alternativa multiplica la cantidad de mensajes en función del tamaño del *batch* y convierete al nodo coordinador en cuello de botella, además de introducir un punto de falla adicional. El anillo, en cambio, distribuye la responsabilidad y mantiene el costo acotado.

En las secciones siguientes, cuando expliquemos cada consulta Q1-Q5 y aparezcan los anillos de control en los diagramas de robustez, se asume que este protocolo garantiza la coherencia del procesamiento.

5.4. Consideraciones frente a la tolerancia a fallos

Al mejorar nuestro sistema para garantizar la tolerancia a fallos, decidimos utilizar colas dedicadas para cada *worker* con sharding determinístico en vez de utilizar colas con consumidores competitivos. Esto da pie a simplificar el sistema de manejo de EOF, potencialmente eliminando los anillos entre réplicas. Sin embargo, decidimos mantenerlos ya que se trata de un sistema funcional ya probado, que permite volver a una implementación de consumo competitivo en el futuro.

6. Vista Física y de Despliegue

En esta sección describimos cómo se materializan los nodos lógicos definidos en el DAG sobre los recursos físicos del sistema. Se presentan, primero, los diagramas de robustez, que muestran las colas, *exchanges* y *workers* concretos involucrados en cada consulta y, posteriormente, el diagrama de despliegue, que sitúa estos componentes sobre los nodos físicos.

6.1. Tipos de nodos lógicos

Los nodos representados en los diagramas se organizan en dos categorías según su responsabilidad dentro del sistema: los **actores**, que participan en la interacción con componentes externos o en la coordinación del flujo principal, y los **workers**, que realizan tareas internas de procesamiento sobre los datos.

6.1.1. Actores

Los actores corresponden a nodos con responsabilidades específicas dentro de la arquitectura, como la recepción de solicitudes del cliente, la comunicación con servicios externos y la obtención de información desde APIs. En esta categoría se incluyen los nodos *Client*, *Gateway* y *Fetcher*.

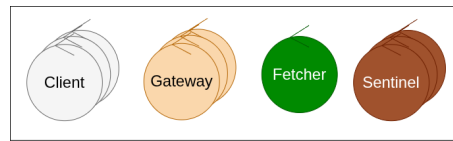


Figura 7: Actores del sistema.

- **Client:** lee los datasets, envía sus transacciones y cuentas al sistema y recibe los resultados consolidados. Es quien inicia y consume cada ejecución.
- **Gateway:** actúa como puerta de entrada y de salida. Desempaqueta los lotes que envía el cliente e inyecta cada transacción o cuenta al flujo interno (con su *ClientId* y *GatewayId*); en sentido inverso, recolecta los resultados finales y los devuelve al cliente que los solicitó.
- **Fetcher:** obtiene de un servicio externo los tipos de cambio del período de interés y los pone a disposición del pipeline, de modo que las consultas puedan convertir los montos a USD (lo requiere Q5).
- **Sentinel:** Monitorea constantemente mediante *heartbeats* a cada uno de los nodos *worker*, a los *gateways* y a sus pares, para comprobar si están vivos. En caso de detectar un nodo muerto, lo reinicia. Este tipo de nodo está explicado más a detalle en la sección 8 Tolerancia a Fallas.

6.1.2. Workers

Todos los workers comparten la misma estructura: consumen tuplas, les aplican una transformación y emiten el resultado, y se diferencian únicamente por la *estrategia* que ejecutan.

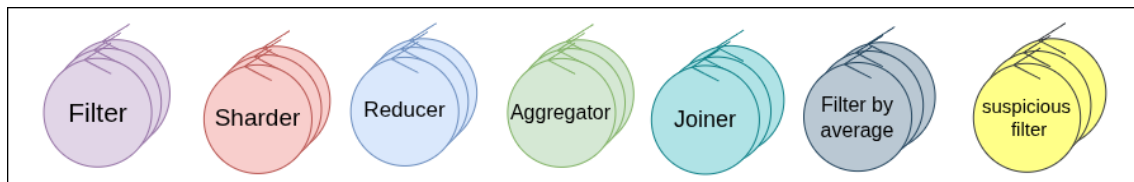


Figura 8: Estrategias de los workers.

- **Filter:** evalúa un predicado sobre cada tupla y la deja pasar o la descarta (por moneda, período, monto o medio de pago).
- **Sharder:** redistribuye las tuplas según una clave (cuenta o banco) para que la etapa siguiente pueda procesarlas.
- **Reducer:** calcula un resultado parcial sobre el subconjunto de tuplas que le toca (un máximo, una suma con su conteo o los caminos entre cuentas).
- **Aggregator:** combina los resultados parciales de los *reducers* en el resultado global de la rama (el máximo global, el promedio o el conteo total).
- **Joiner:** une dos flujos relacionados en uno solo; por ejemplo, cruza el máximo de cada banco con su nombre, o consolida respuestas de un cliente.
- **Filter by average:** variante del filtro propia de Q3 que conserva únicamente las transacciones cuyo monto es inferior a la centésima parte del promedio previamente calculado.
- **Suspicious filter:** pre-filtro propio de Q4 que conserva únicamente las cuentas con un mínimo de contrapartes distintas (puntos de entrada o de salida) y descarta el resto antes de reconstruir los caminos.

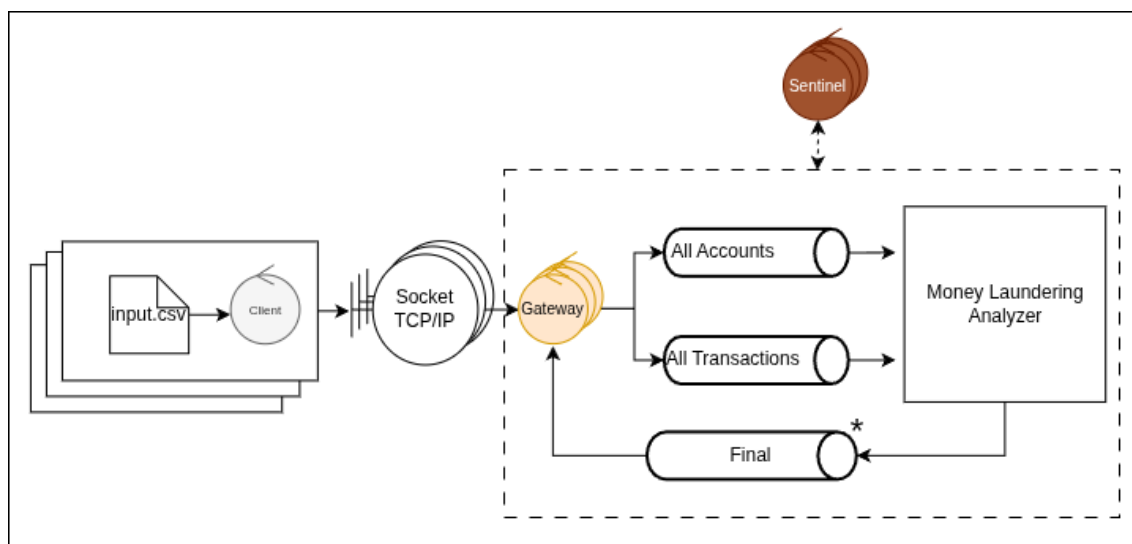
Es importante aclarar que los nodos de entrada de cada consulta (sean Sharder / Filter o Reducers) son los encargados de la coordinación del EOF mediante la topología de anillo (descrito en la sección *Manejo de EOF*) y de aplicar la clave de sharding correspondiente para derivar los datos a las etapas posteriores.

6.2. Diagrama de Robustez

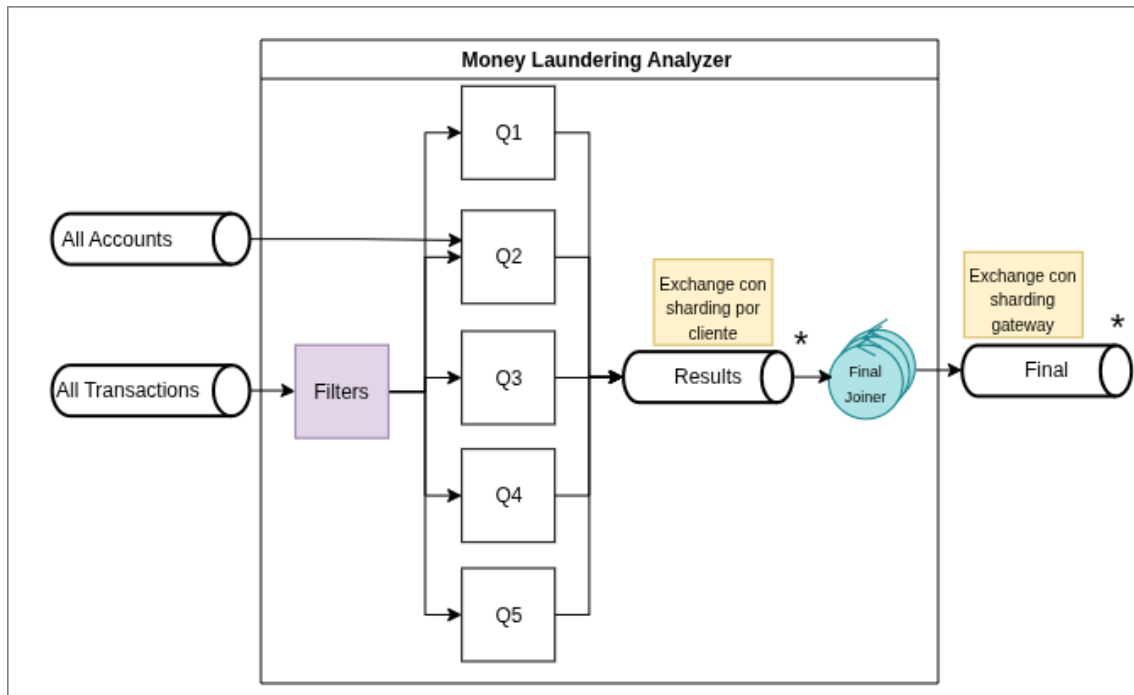
A continuación se presenta la vista general del sistema y, posteriormente, el detalle de cada consulta Q1–Q5. La nomenclatura es uniforme en todos los diagramas: los cilindros representan colas o *exchanges* de RabbitMQ, los controladores encerrados en un cuadrado punteado en fucsia representan grupos de *workers* idénticos coordinados mediante topología de anillo (cuyo protocolo se describió en la sección anterior).

Por su parte, el asterisco (*) sobre una cola representa un exchange direct con múltiples colas, una por cada instancia del worker siguiente. Estas colas son alimentadas por un worker previo al exchange, que aplica una función de hash sobre una clave y la reduce sobre módulo la cantidad de particiones. Por ser determinística, una misma clave cae siempre en la misma cola. A la entrada del sistema esa clave es el propio *batch* que envía el cliente, con lo que el *gateway* reparte la carga entre las instancias de la primera etapa. En las etapas posteriores la clave es un atributo de dominio, como el ID del cliente, el banco o la cuenta. Esto garantiza que todos los eventos asociados a una misma entidad sean enrutados y encolados siempre en la misma partición. Por otro lado, la estrella (★) representa que se trata de un exchange fanout.

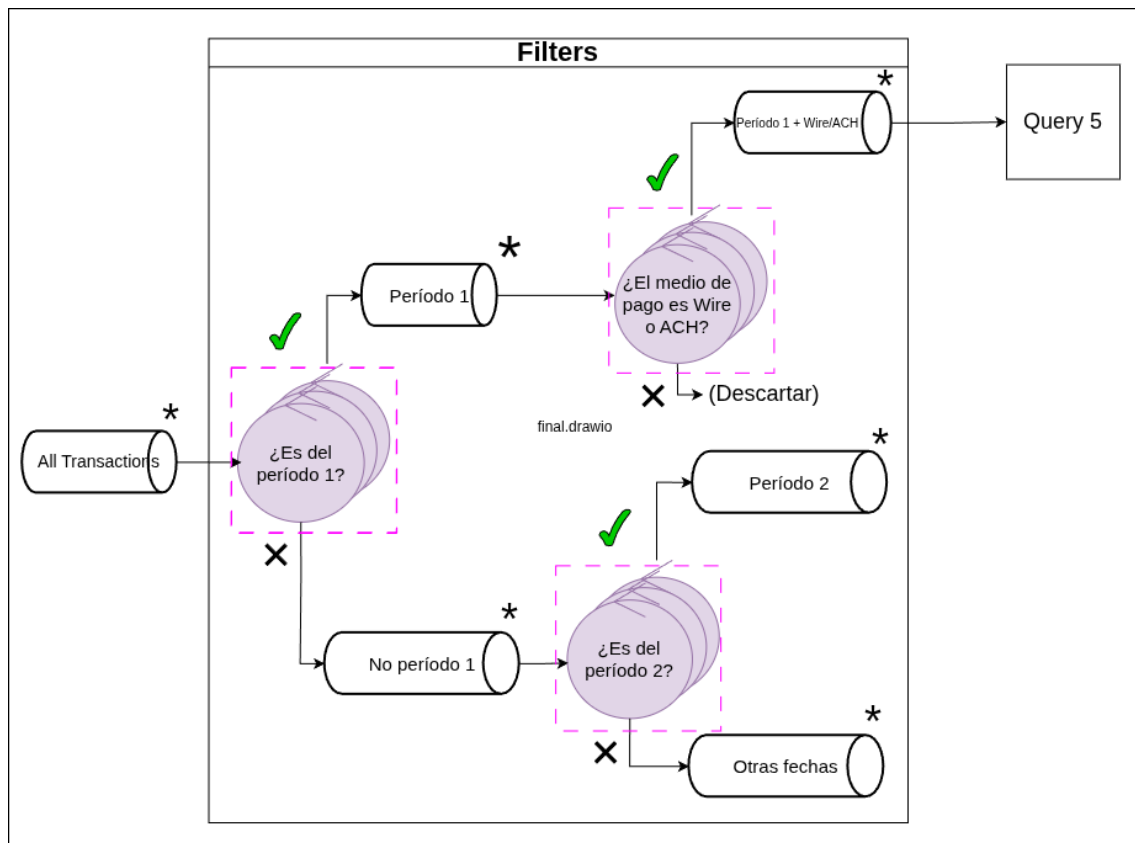
- **Vista general.** El cliente se conecta al *Gateway* mediante un socket TCP/IP, y envía las transacciones y las cuentas bancarias. El *gateway* encola estos datos en las colas *All Transactions* y *All Accounts* respectivamente. El sistema procesa los datos, genera los resultados de cada consulta y los encola en *Final* shardeado por cliente, de forma que el *Gateway* del mismo cliente los reciba y se los devuelva. El conjunto de nodos de procesamiento, junto con los *gateways*, comunican un *healthcheck* al clúster de nodos *Sentinel*. Este tipo de nodo está explicado más a detalle en la sección 8 Tolerancia a Fallas.

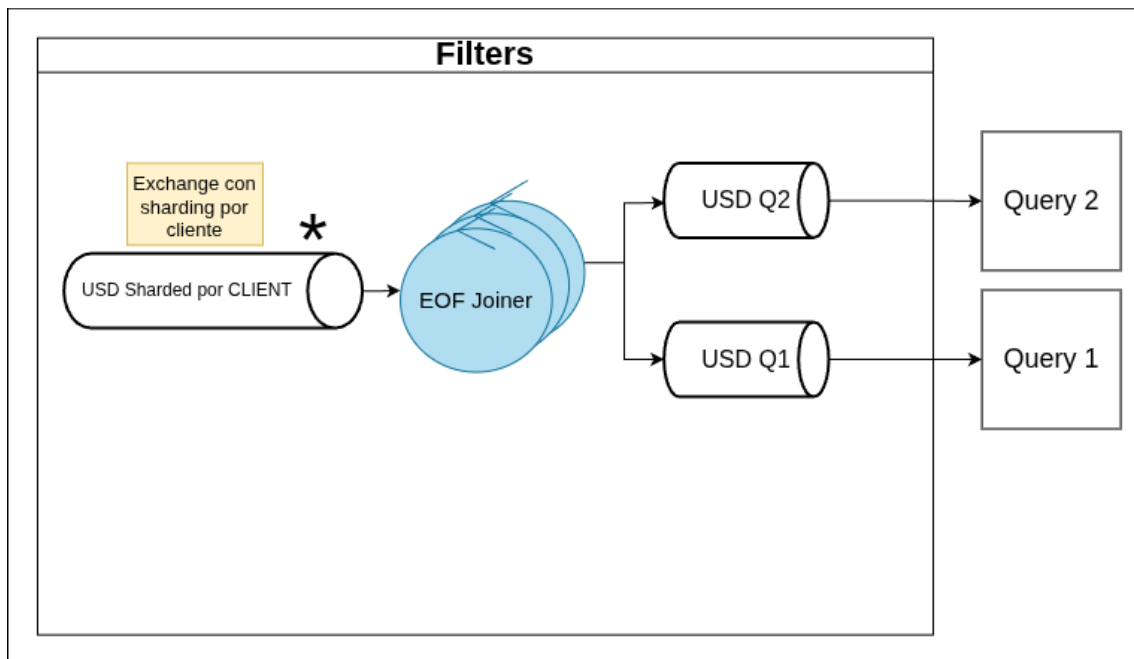
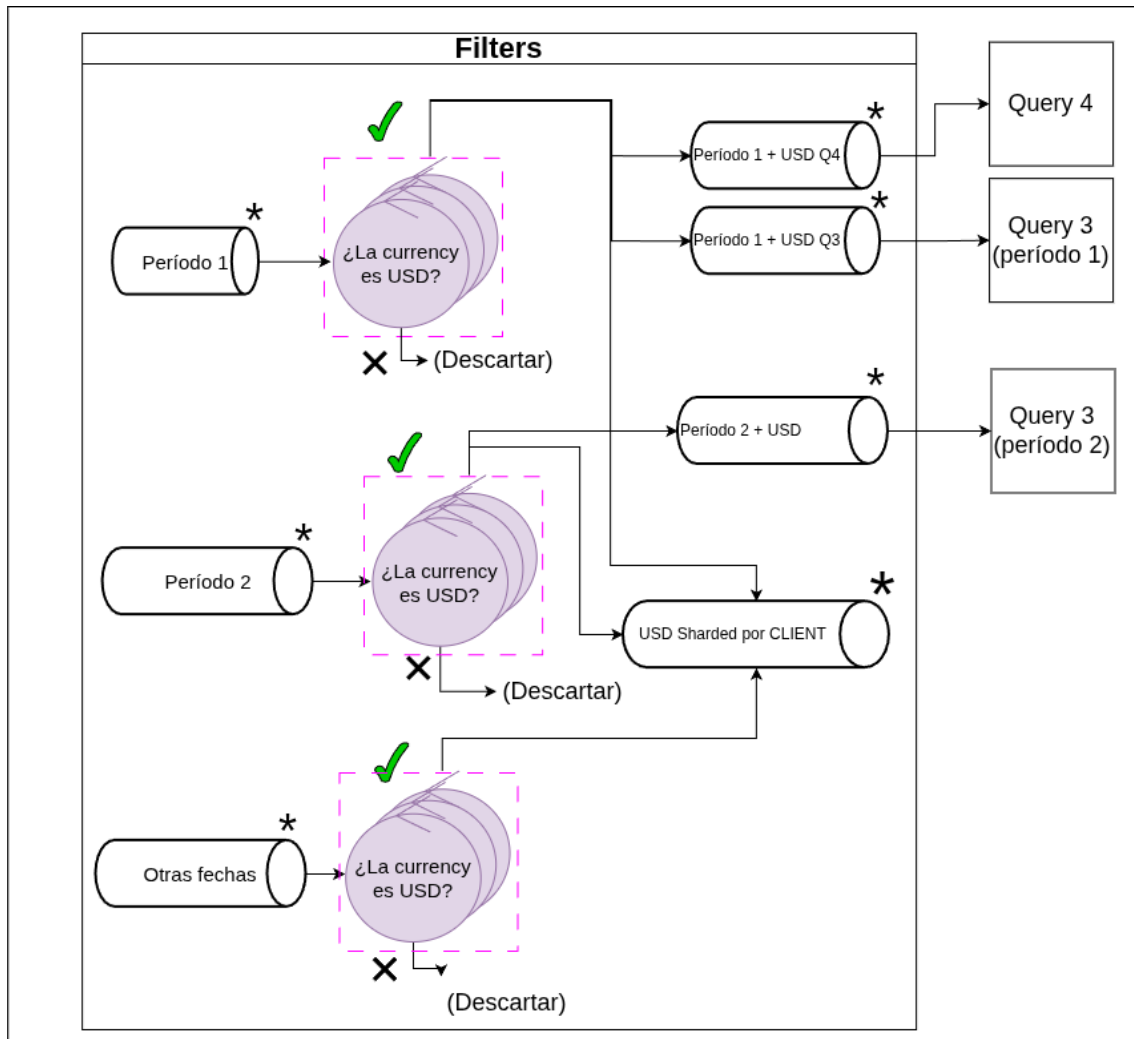


- **Vista general de los nodos de procesamiento.** Un conjunto de nodos *Filter* se encarga de filtrar y descartar columnas de las transacciones de acuerdo a lo necesitado por cada una de las consultas. Los datos llegan mediante colas a cada uno de los grupos de nodos de las consultas (Q1 a Q5). El *Final Joiner* recibe los resultados de todas las consultas desde el exchange *Results* y los rutea a la cola del gateway correspondiente a cada cliente. Además, consolida los múltiples EOF que emiten las réplicas de cada query en un único EOF por consulta, señalizando al gateway que esa query finalizó para ese cliente.



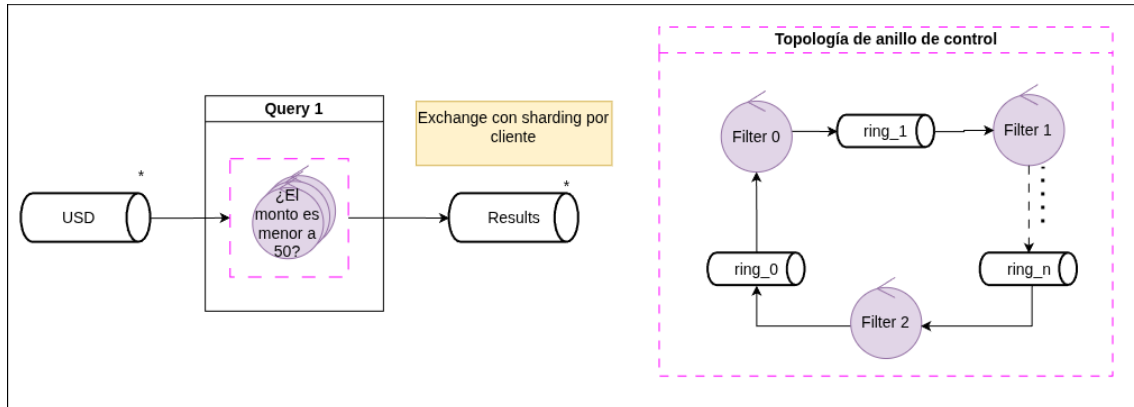
- **Filtros.** Cada grupo de nodos filtro recibe transacciones, con cada réplica consumiendo de su propia cola dedicada, y posee dos conjuntos de middlewares de salida. Para cada una de las transacciones, evalúa un predicado y según el resultado (booleano) la encola en los middlewares de un conjunto de salida u del otro. En algunos grupos de *Filters*, el conjunto de salida correspondiente al no cumplimiento del predicado está vacío, por lo que las transacciones que llegan hasta ahí se descartan. Los últimos grupos de *Filters* encolan las transacciones en los mismos middlewares que consumen los nodos de procesamiento de cada consulta. Debido a que las consultas tienen condiciones de filtrado que son compartidas, la topología de nodos *Filter* no es puramente la de un árbol, sino que en un punto se vuelven a juntar flujos separados (más específicamente, en el exchange *USD Sharded por client*, por donde pasan todas las transacciones en dólares). Esta particularidad requiere agregar un grupo de nodos *EOF Joiner*, los cuales se encargan de unir los tres mensajes de EOF provenientes de los flujos convergentes, y generar un único mensaje combinado con la suma de los tres. Para facilitar la lectura, se dividió el diagrama de robustez de esta parte del sistema en tres:





- **Query 1 (Micropagos).** Por su simplicidad, esta consulta se resuelve con un único grupo

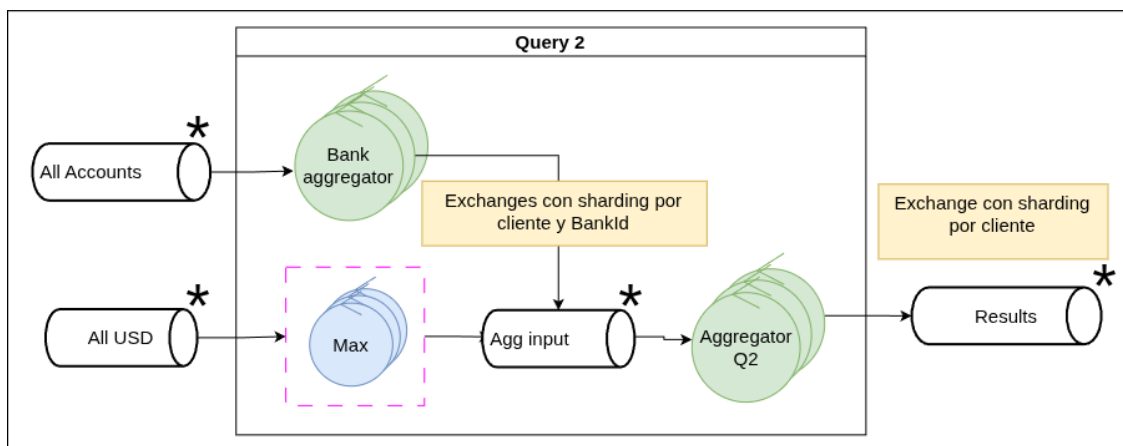
de *workers* idénticos (*Filtros*) en topología de anillo. Cada réplica consume de su propia cola dedicada (con transacciones en USD ya filtradas por la cadena común), conserva únicamente las de monto inferior a 50, proyecta la tupla y la encola en **Results**. El recuadro a la derecha del diagrama explicita la topología del anillo de control: cada réplica i consume de su cola privada $ring_i$ y publica a $ring_{(i+1) \bmod n}$ para coordinar el cierre por EOF descrito previamente.



- **Query 2 (Máximos por Entidad).** La consulta Q2 implementa un esquema de reducción local seguido de una etapa de agregación con *join*. El procesamiento se divide en dos ramas que operan en paralelo. Por un lado, los nodos Max_i consumen las transacciones en USD y mantienen, para cada banco, el monto máximo observado localmente. Al finalizar el procesamiento del cliente, envían sus resultados parciales al *exchange q2_agg_input*.

En paralelo, los nodos $Bank_Aggregator_i$ procesan el archivo de cuentas y construyen un diccionario que asocia cada **bankID** con el nombre correspondiente del banco. Estos parciales también son enviados al mismo *exchange* una vez recibido el EOF.

Ambas ramas distribuyen sus mensajes utilizando *sharding* por (**clientID**, **bankID**), asegurando que un mismo nodo $Aggregator_i$ reciba tanto los máximos calculados como la información descriptiva del banco asociado. Cuando el agregador recibe todos los EOFs esperados —tres provenientes de los nodos Max y tres de los $Bank_Aggregator$ — realiza el *join* entre ambas fuentes, reemplaza los identificadores por los nombres de banco correspondientes y publica el resultado final en **Results**.



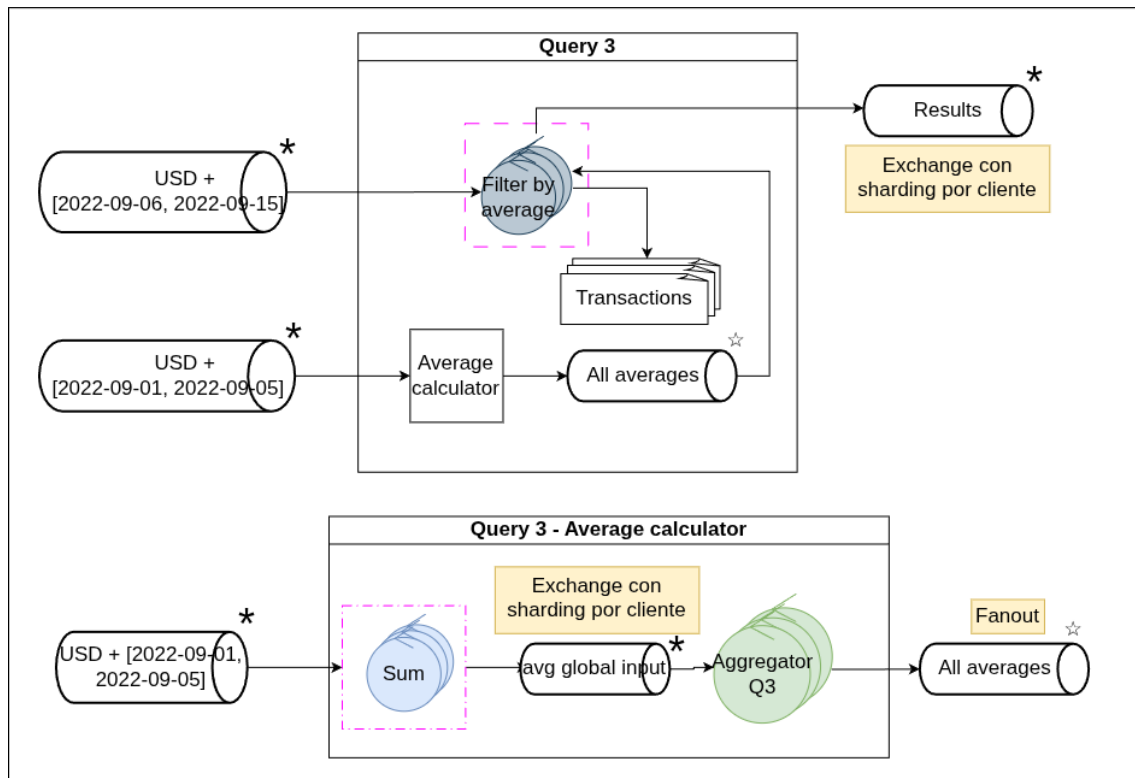
- **Query 3 (Anomalías Históricas).** Esta consulta se organiza en dos sub-ramas que convergen y se sincronizan en el componente *Filter By Average*.

La primera sub-rama tiene como objetivo calcular el promedio global por formato de pago. Para ello, los nodos Sum_i —un conjunto de *workers* idénticos coordinados en anillo—

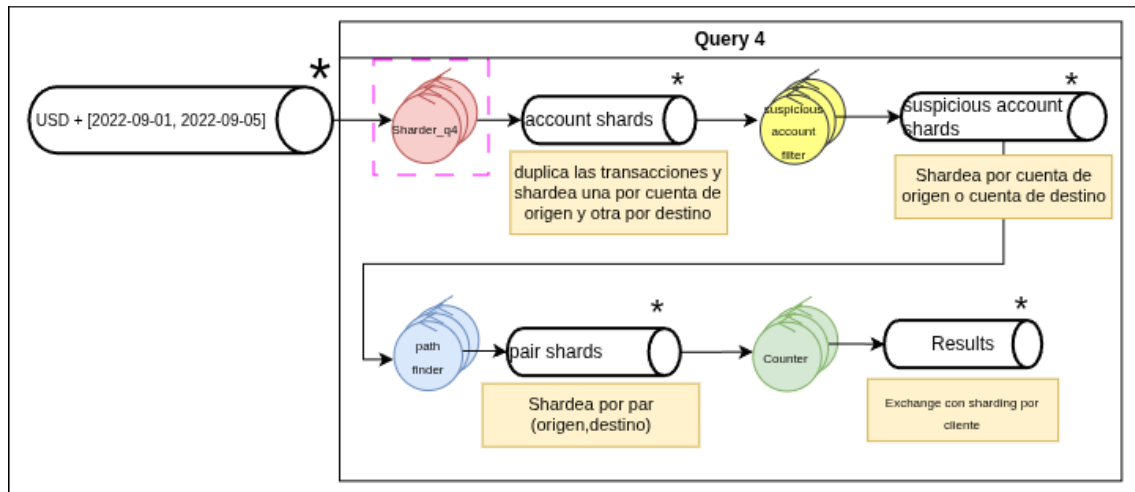
consumen las transacciones en USD correspondientes al período 1 y mantienen, para cada formato de pago, una suma acumulada y un conteo parcial. Al finalizar el procesamiento del cliente, emiten dichos parciales al *exchange Partial sums*, utilizando *sharding* por cliente. Luego, cada *Aggregator Q3-i* consume los parciales asignados, calcula el promedio final por formato de pago y difunde esos resultados a todos los nodos *Filter By Average* mediante un mecanismo de *broadcast*.

La segunda sub-rama se encarga de filtrar las transacciones correspondientes al período 2. Cada *Filter By Average* consume simultáneamente dos flujos: las transacciones USD y los promedios distribuidos desde el *exchange Averages*. Si las transacciones llegan antes de que los promedios estén completos, estas se persisten temporalmente en disco; en cambio, una vez disponibles los promedios, las nuevas transacciones pueden filtrarse en línea a medida que arriban.

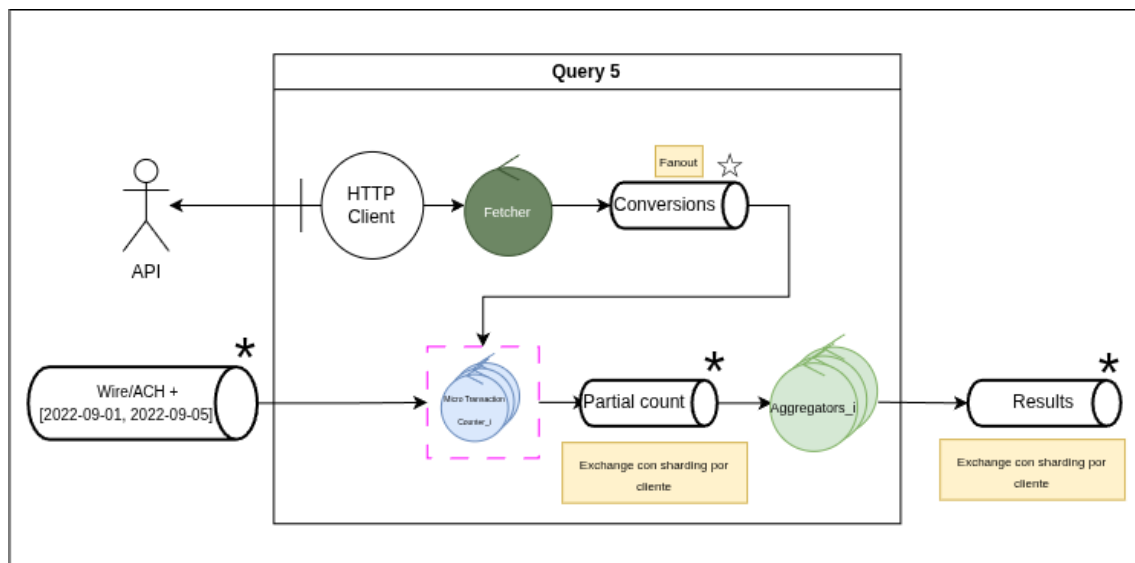
Cuando el componente recibe tanto el EOF del flujo de transacciones como el EOF asociado a los promedios, recorre el buffer persistido en disco y conserva únicamente aquellas transacciones cuyo monto resulta inferior al 1 % del promedio correspondiente a su formato de pago. Finalmente, las transacciones filtradas son emitidas hacia *Results*.



- Query 4 (Scatter-Gather).** La consulta está compuesta por cuatro etapas en *pipeline*. El *Sharder-i* consume las transacciones USD del período 1 y emite cada una duplicada al *exchange Account Shards*: una copia *shardeada* por cuenta de origen y otra por cuenta de destino. El *Suspicious Account Filter-i* consume de su shard de *Account Shards* y detecta cuentas con cinco o más contrapartes distintas; al superarse el umbral, emite todas las transacciones acumuladas de esa cuenta ahora *shardeadas* por la contraparte al *exchange Suspicious Account Shards*. El *Path Finder-i* consume de *Suspicious Account Shards* y reconstruye caminos de tres saltos $A \rightarrow M \rightarrow B$ a través de cuentas intermediarias sospechosas, emitiendo cada conjunto al *exchange Pair Shards* *shardeado* por el par (A, B) . Finalmente, el *Counter-i* acumula las cuentas intermediarias distintas por par (A, B) y emite a *Results* aquellos pares que superen las cinco intermediarias. El detalle del algoritmo del *Path Finder* y un ejemplo de ejecución se presentan en la *Vista de Procesos*.



- **Query 5 (Micropagos Internacionales).** La consulta incorpora una particularidad respecto a las anteriores: depende de un servicio externo. El nodo único *Fetcher* consulta, vía cliente HTTP, la API de Frankfurter para obtener y enviar via fanout las cotizaciones diarias necesarias para la conversión a USD. El grupo de nodos *Micro Transaction counter* desencola esos valores hasta asegurarse de haber obtenido todos. Una vez hecho esto cambia su cola de input y desencola transacciones convirtiendo su monto a dólares. Se incrementa un contador parcial sólo si el monto resulta menor a 1 USD. Al cierre del cliente, los parciales se emiten al *exchange Partial count*, y el *Aggregator_i* consolida la suma final por cliente y la publica en *Results*.



Decidimos realizar una única consulta a la API al iniciar el nodo *Fetcher* y almacenar los resultados en memoria en cada *Converter*. Ya que pudimos observar que es factible almacenar todos los resultados en memoria, dado que los volúmenes de datos son razonables. Por ejemplo, una consulta para la conversión de todas las monedas a USD durante un período de un año ocupa aproximadamente 3.5 MB, mientras que para el período solicitado en el trabajo práctico el tamaño es de alrededor de 40 kB.

Como se observa:

```
1 $ curl -H "Accept: application/x-ndjson" "https://api.frankfurter.dev/v2/rates?from=2022-01-01&to=2022-12-31&base=USD" -o
  rates_2022.ndjson
2 % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
3    Dload  Upload   Total   Spent    Left   Speed
4 100 3067k    0 3067k    0     0    275k    0 --:--:--  0:00:11 --:--:-- 359k
```

Listing 1: Estadísticas de la solicitud a la API para un período de un año entero

```

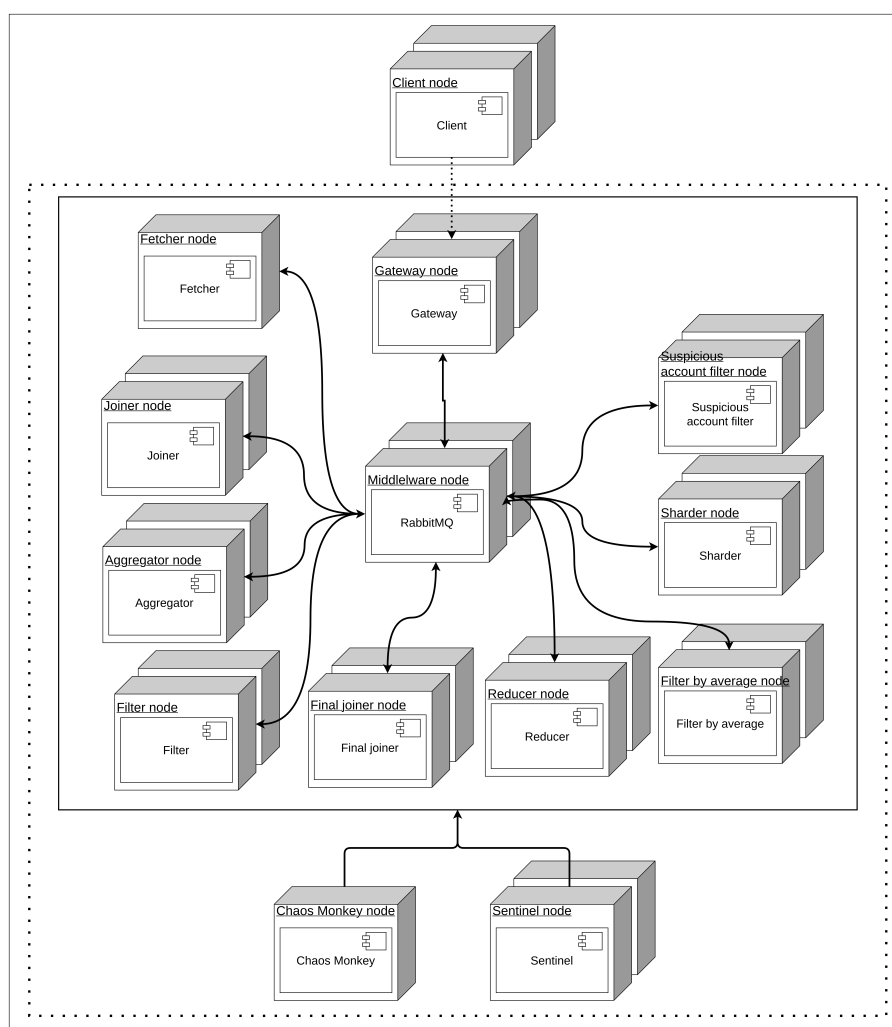
1 $ curl -H "Accept: application/x-ndjson" "https://api.frankfurter.dev/v2/rates?from=2022-09-01&to=2022-09-05&base=USD" -o
   rates_period.ndjson
2 % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
3           %             Dload  Upload  Total   Spent    Left   Speed
4 100 39708    0 39708    0    0 32261    0 --:--:--  0:00:01 --:--:-- 32256

```

Listing 2: Estadísticas de la solicitud a la API para el período solicitado en el trabajo práctico

6.3. Diagrama de Despliegue

El siguiente diagrama presenta la organización física de los componentes sobre nodos de cómputo. Cada nodo se ejecuta en un contenedor Docker independiente y toda la comunicación entre ellos atraviesa el nodo de *middleware*, que aloja el *broker* RabbitMQ. Esta indirección provee la transparencia que requiere un sistema basado en grupos, donde los *workers* no se conocen entre sí, sino que se acoplan únicamente a las colas y *exchanges* sobre los que producen o consumen.



Los paquetes representados en el diagrama corresponden a las categorías definidas en la sección *Tipos de nodos*. Los **actores** se ubican en los extremos de la arquitectura y son responsables de la interacción con componentes externos. El nodo *Client* establece comunicación con el *Gateway* mediante sockets TCP/IP, mientras que el *Gateway* actúa como punto de enlace entre los clientes y el *middleware*, encargándose tanto de recibir y desempaquetar los lotes de mensajes como de retornar los resultados procesados. Por su parte, el nodo *Fetcher* obtiene información desde APIs externas y publica las cotizaciones en RabbitMQ.

El resto de los componentes corresponde a los **workers** (*Filter*, *Sharder*, *Reducer*, *Aggregator*, *Joiner*, *Final joiner*, *Filter by average* y *Suspicious account filter*), cada uno desplegado en un nodos independientes y comunicados exclusivamente a través del *middleware*. Cada grupo de *workers* se escala horizontalmente de forma independiente, replicando su contenedor y sumándolo si corresponde al anillo de control de su grupo (descrito en *Manejo de EOF*). Por simplicidad gráfica, los nodos de un mismo tipo se agrupan aunque no ejecuten exactamente la misma tarea en cada consulta (por ejemplo, los *aggregators* de Q2, Q3 y Q5 realizan estrategias de agregación distintas).

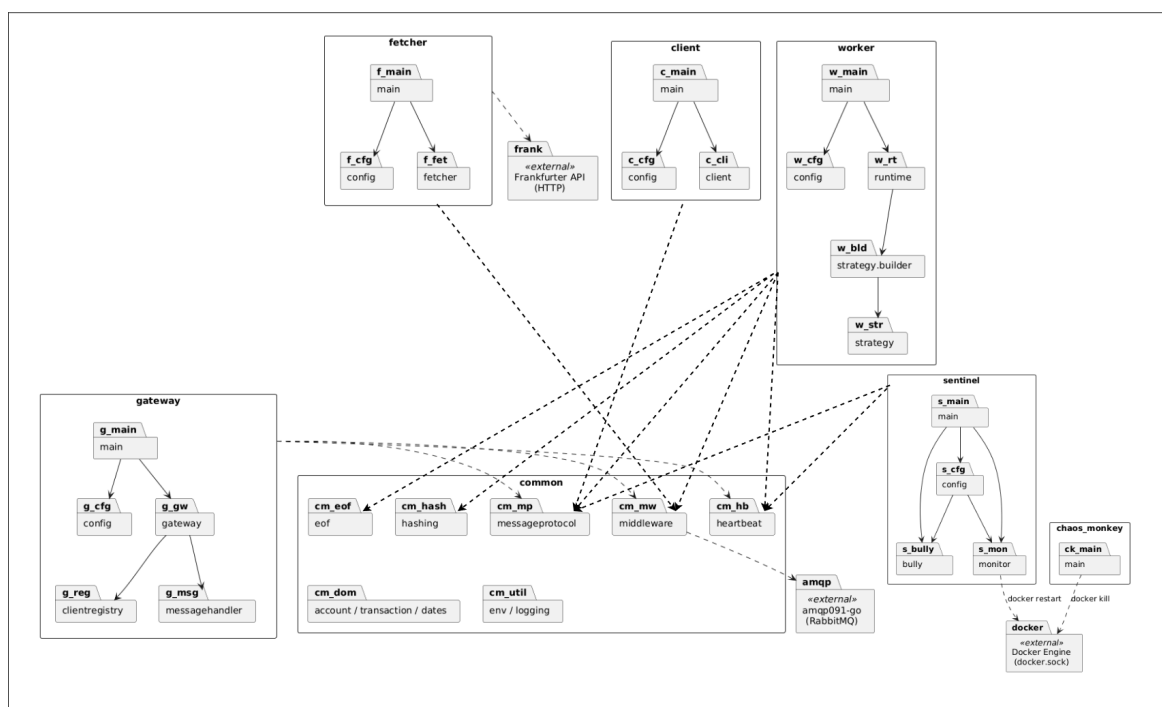
Por fuera del plano de datos descrito hasta aquí se despliega el subsistema de tolerancia a fallas, también sobre contenedores Docker independientes. El *Chaos Monkey* inyecta fallas terminando procesos al azar, mientras que un conjunto de N *centinelas* idénticos detecta la caída de los demás nodos y los repone. A diferencia del resto de los contenedores, estos dos no se acoplan al *broker* de RabbitMQ, ya que montan el *socket* de Docker del *host* para poder terminar y reiniciar contenedores, y se coordinan entre sí por canales propios ajenos al *middleware*. Su funcionamiento se detalla en la sección *Tolerancia a fallas*.

7. Vista de Desarrollo

En esta sección presentamos la organización del código fuente del sistema, agrupado en paquetes que se corresponden con las unidades de despliegue y con los componentes transversales reutilizados por más de una unidad.

7.1. Diagrama de Paquetes

El código se estructura en seis unidades desplegables — *client*, *gateway*, *worker*, *fetcher*, *sentinel* y *chaos_monkey* — que se apoyan en una biblioteca compartida, *common*. El diagrama se derivó directamente del grafo de *imports* del módulo, a nivel de paquete, y se presenta en dos partes: la primera con *client*, *worker*, *fetcher*, *sentinel* y *chaos_monkey*; la segunda con *gateway* y *common*. Las unidades de la primera figura dependen de *common* (segunda figura) según se detalla a continuación.



- **client.** Su *main* delega en el paquete *config* y *client*, que lee los datasets de entrada, envía los *batches* al *gateway* y recibe los resultados consolidados. Usa de *common* los paquetes *account*, *transaction* y *messageprotocol/external* (protocolo TCP con el *gateway*).
- **gateway.** Su *main* delega en el paquete *gateway*, que coordina *config*, *clientregistry* (asociación clientesocket) y *messagehandler* (desempaqueta los *batches* entrantes y publica cada transacción o cuenta al *middleware*). Usa de *common* *messageprotocol* (externo e interno), *middleware*, *heartbeat*, *account* y *transaction*.
- **worker.** Su *main* delega en *runtime*, el bucle genérico (consumir → transformar → emitir) que despacha hacia *strategy* y su *builder*. *strategy* concentra la lógica concreta de cada Q1–Q5 y de los auxiliares (*sharder*, *suspicious filter*, etc.), siguiendo el patrón *Strategy*. Usa de *common* *eof*, *hashing*, *heartbeat*, *messageprotocol* (interno y externo) y *middleware*.
- **fetcher.** Su *main* delega en el paquete *fetcher*, que consulta por HTTP la API de Frankfurt para obtener las cotizaciones del período y las publica al *middleware* para que Q5 convierta los montos a USD.
- **sentinel.** Su *main* levanta el monitor que vigila la salud del resto de los nodos. Detecta caídas a partir de los *heartbeats* que recibe, elige un líder mediante el algoritmo Bully y, cuando corresponde, repone los contenedores caídos con *docker restart*. Usa de *common* los paquetes *heartbeat* (decodificación de las señales de vida), *messageprotocol*, *env* y *logging*.
- **chaos_monkey.** Su *main* ejecuta un bucle que, cada cierto intervalo, termina un contenedor al azar con *docker kill* para inyectar fallas y ejercitar la recuperación. Usa de *common* únicamente *env* y *logging*.
- **common.** Biblioteca transversal reutilizada por todas las unidades. Agrupa *messageprotocol* —con *external* (safeio para el socket y *serializer* para el formato del protocolo TCP clientegateway) e *inner* (mensajes internos sobre RabbitMQ)—, *middleware* (abstrae AMQP), *eof* (protocolo de cierre en anillo), *hashing* (*sharding*), *heartbeat* (señales de vida de los nodos hacia los centinelas) y los paquetes de dominio y utilidad *account*, *transaction*, *dates*, *env* y *logging*.

Concentrar lo compartido en *common* evita duplicar código entre unidades y aísla las decisiones tecnológicas en un único lugar. De esta forma, RabbitMQ queda contenido en *common/middleware*, el cliente HTTP en el *fetcher* y la interacción con el motor de Docker en los *centinelas* y el *chaos monkey*.

8. Tolerancia a fallas

Hasta aquí el sistema fue descrito asumiendo que los nodos de cómputo permanecen en pie durante toda la corrida. En un entorno multicomputadora un proceso podría morir en cualquier instante por un error de software, una falla de hardware o, simplemente, porque el orquestador lo reinicia. Tolerar fallas significa, en este contexto, que la caída abrupta de cualquier nodo de cómputo no comprometa el sistema, que debe detectar la caída, reponer el nodo y converger al mismo resultado que habría producido sin la falla.

El problema se ataca en tres frentes complementarios. Primero, un **Chaos Monkey** inyecta fallas de forma deliberada para *validar empíricamente* que la tolerancia funciona. Segundo, un **cluster de centinelas** se encarga de *detectar* qué nodos cayeron y *revivirlos*. Tercero, una vez que un nodo revive, los mecanismos de **persistencia** y **deduplicación** garantizan que el reprocesamiento no pierda ni duplique datos. Las dos primeras partes hacen que un proceso muerto vuelva a estar vivo y, la última, que volver a la vida no corrompa el resultado.

8.1. Inyección de fallas: Chaos Monkey

El Chaos Monkey es un nodo cuya única responsabilidad es matar procesos al azar. Cada cierto intervalo configurable (por defecto, cada 20 segundos) elige un contenedor de una lista de candidatos y lo termina con `docker kill`, lo que equivale a enviarle una señal `SIGKILL`.

La elección de `SIGKILL` no es casual: es la forma más violenta de terminar un proceso, ya que no le concede ninguna oportunidad de hacer una limpieza ordenada ni de *flush* su estado. El proceso muere en la instrucción que estuviera ejecutando, sea cual fuere. El razonamiento de diseño es que, si el sistema converge al resultado correcto bajo el peor escenario posible, que es una muerte súbita en cualquier punto del código, entonces tolera con holgura cualquier modo de falla más benigno.

Cabe mencionar que el *broker* RabbitMQ queda siempre excluido de la lista de víctimas. Este es el medio de comunicación sobre el que se apoya toda la arquitectura y, a los fines de este trabajo, se lo asume confiable. Todos los demás procesos son candidatos, incluidos los propios centinelas.

8.2. Detección y recuperación: cluster de centinelas

La detección y recuperación de nodos caídos está a cargo de un cluster de N **centinelas idénticos**, siendo N un número entero mayor que cero, cada uno en su propio contenedor. Con un único centinela ($N = 1$) ese nodo sería un punto de falla que, al caer, dejaría al sistema entero sin mecanismo de recuperación y por ende es indispensable que en la práctica se desplieguen varias réplicas.

8.2.1. Detección por heartbeat

Cada *worker* emite periódicamente un *heartbeat* (una señal de vida) hacia las N réplicas de centinela. El *heartbeat* se envía cada cinco segundos, con un *jitter* aleatorio que desincroniza a los distintos emisores para evitar ráfagas simultáneas. Un centinela considera caído a un *worker* cuando transcurre un umbral de tiempo (por defecto, veinte segundos) sin haber recibido ningún *heartbeat* suyo. Como ese umbral abarca varios períodos de emisión, la pérdida ocasional de un *heartbeat* aislado no produce un falso positivo. Además, durante un período de inicial (treinta segundos) los *workers* que todavía no emitieron ningún *heartbeat* no se reinician, para dar tiempo al arranque del sistema.

Una decisión a aclarar de este diseño es que la salud de un *worker* se determina **exclusivamente** a partir de la presencia o ausencia de su *heartbeat*. Los centinelas nunca interrogan al motor de Docker (no ejecutan `docker ps` ni `docker inspect`).

8.2.2. Recuperación

Cuando un centinela detecta una caída, repone el nodo mediante `docker restart`. El contenedor del centinela monta el *socket* de Docker del *host* y, a través de él, ordena reiniciar al contenedor caído. Se impone un *cooldown* (sesenta segundos) entre dos reinicios consecutivos del mismo contenedor, para evitar entrar en un bucle de reinicios si un nodo falla de forma repetida apenas arranca.

Además, si los N centinelas detectan la misma caída de forma independiente, todos ellos intentarían reiniciar al mismo nodo, provocando reinicios redundantes y potencialmente dañinos. Para evitarlo, **solo el líder ejecuta los reinicios**; los restantes detectan las caídas pero permanecen a la espera. De este modo, la detección está replicada (los N centinelas escuchan los *heartbeats*) pero la acción correctiva está centralizada en un único responsable. El problema se reduce, entonces, a elegir y mantener ese líder de forma robusta frente a la caída del propio líder.

8.2.3. Elección de líder: algoritmo Bully

Para elegir al líder se implementó el algoritmo **Bully**. La idea del algoritmo es que cada centinela tiene asignado un identificador único y la regla de oro es que *el de mayor identificador debe ser el líder*. Cuando un centinela detecta que el líder dejó de responder, convoca una elección enviando un mensaje de **Election** a todos los nodos con identificador mayor al suyo. Si alguno de ellos contesta (OK), el convocante se retira y espera, porque sabe que hay alguien con más prioridad que se hará cargo. Si, en cambio, nadie responde dentro de un tiempo de espera, el convocante se proclama líder y lo anuncia al resto con un mensaje de **Coordinator**. La elección se dispara tanto al arrancar el cluster como cada vez que un centinela deja de percibir señales de vida del líder durante demasiado tiempo.

¿Por qué Bully y no otro algoritmo? El problema que se debe resolver es de alcance acotado, ya que solo se requiere elegir un único coordinador dentro de un grupo pequeño y conocido. La alternativa más cercana sería un algoritmo de elección en anillo, pero este exige mantener una topología de anillo viva y reconstruirla cada vez que un nodo cae, que es justamente el evento frecuente en nuestro escenario. Bully, en cambio, opera sobre una membresía estática y conocida de antemano, por lo que no necesita reparar ninguna estructura tras una caída. Su principal objeción es el costo en mensajes, del orden de $O(N^2)$ en el peor caso, donde N es la cantidad de centinelas, pero mientras esa cantidad se mantenga pequeña, como ocurre aquí, donde bastan unas pocas réplicas para garantizar la redundancia, ese costo resulta irrelevante y el argumento que normalmente penaliza a Bully deja de aplicar a esta escala.

Una **precondición** del algoritmo Bully es que el canal de comunicación entre los nodos sea confiable: la pérdida de un mensaje de OK o de **Coordinator** podría dejar a más de un centinela creyéndose líder simultáneamente (*split-brain*). Esta precondición es la que determina, como se verá más adelante, qué protocolo de transporte usan los centinelas para coordinarse.

8.2.4. Recuperación de los propios centinelas

El mismo mecanismo de *heartbeat* que vigila a los *workers* opera también puertas adentro del cluster. Cada centinela emite su propia señal de vida hacia los demás, con una cadencia bastante más alta que la de los *workers*, del orden de una por segundo, y por eso el umbral para darlo por caído es también mucho más breve, de apenas unos pocos segundos. Por un lado, este flujo les permite a los seguidores advertir que el líder dejó de emitir y, en consecuencia, convocar una nueva elección. Por otro, le permite al líder advertir la caída de cualquier otro centinela y reponerlo, exactamente igual que repone a un *worker*, con un **docker restart**.

Esto vuelve al cluster capaz de repararse a sí mismo, incluso cuando el que cae es el líder. Si muere un seguidor, el líder lo reinicia. Si muere el líder, sus *heartbeats* se interrumpen, los seguidores lo detectan y eligen un nuevo líder entre los que siguen en pie, y ese nuevo líder, al revisar el estado del cluster, encuentra que su antecesor lleva demasiado tiempo sin dar señales de vida y lo repone.

Para que esta autorreparación no se vuelva en contra hay un equilibrio temporal que respetar. El peligro a conjurar es que las caídas se acumulen más rápido de lo que el cluster alcanza a reponerlas, hasta quedarse sin ningún centinela en pie. Reponer un centinela lleva un tiempo que llamaremos T_{rep} y que se descompone en tres etapas: lo que tarda su *heartbeat* en vencer y darlo por caído (T_{det}), la elección de un nuevo líder en el caso de que el caído fuese el coordinador (T_{elec}) y el reinicio efectivo del contenedor (T_{rein}),

$$T_{\text{rep}} = T_{\text{det}} + T_{\text{elec}} + T_{\text{rein}}$$

Tras reponer un centinela se le aplica además un tiempo de enfriamiento T_{enf} , durante el cual no se lo vuelve a reiniciar. Si notamos con N la cantidad de centinelas y con T_{caos} el intervalo con que el Chaos Monkey mata un proceso, la condición que mantiene al cluster siempre con al menos un centinela vivo es

$$T_{\text{rep}} < T_{\text{enf}} < N \cdot T_{\text{caos}}$$

Cada desigualdad cubre un riesgo distinto. La cota inferior, $T_{\text{rep}} < T_{\text{enf}}$, evita el bucle de reinicios, ya que el enfriamiento debe durar más que la propia reposición para que, cuando expire, el nodo ya haya arrancado y vuelto a emitir su *heartbeat* y no se lo reinicie otra vez creyéndolo aún caído. La cota superior, $T_{\text{enf}} < N \cdot T_{\text{caos}}$, evita el colapso, porque en el peor caso el caos mata una réplica por intervalo y entonces tarda al menos $N \cdot T_{\text{caos}}$ en bajar a las N . Si el enfriamiento superara ese techo, un nodo recién repuesto podría quedar bloqueado por su propio enfriamiento mientras el caos liquida a las restantes, y las caídas se acumularían hasta el colapso total. Mientras el enfriamiento se mantenga por debajo de ese techo, siempre hay reposiciones completándose entre dos muertes y nunca se llega a cero.

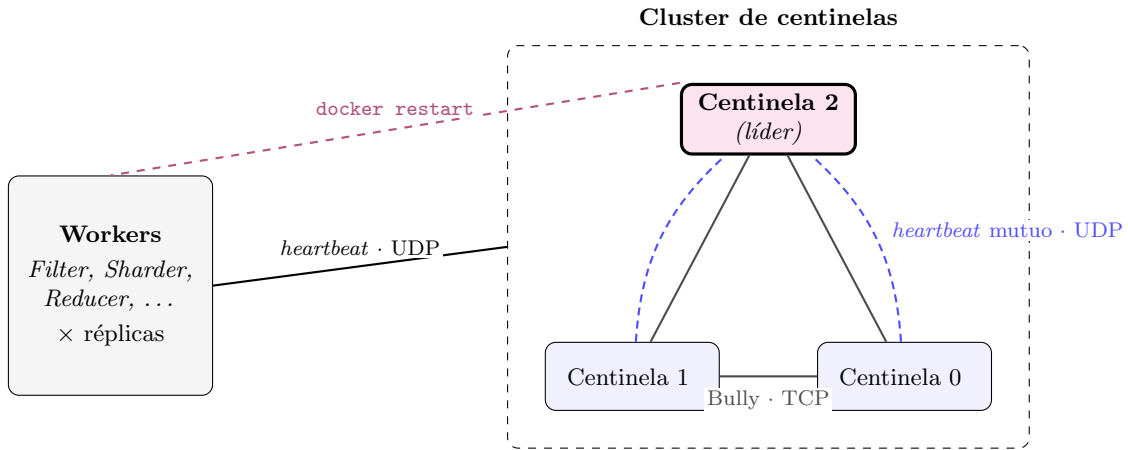
Demostramos este desarrollo con los siguientes valores que usa el escenario de prueba más exigente, en el que el Chaos Monkey ataca únicamente a los centinelas:

$$\underbrace{(3 + 3 + 3) \text{ s}}_{T_{\text{rep}} \approx 9 \text{ s}} < \underbrace{12 \text{ s}}_{T_{\text{enf}}} < \underbrace{3 \cdot 10 \text{ s}}_{N \cdot T_{\text{caos}}} = 30 \text{ s}$$

Es decir, el cluster repone un centinela en unos nueve segundos, espera doce antes de poder volver a tocarlo y dispone de treinta antes de que el caos pudiera siquiera intentar agotar las tres réplicas. El margen es todavía mayor, porque este cálculo supone que cada muerte recae sobre un centinela, cuando en un escenario normal el Chaos Monkey reparte sus golpes entre todos los procesos del sistema y el ritmo efectivo de bajas sobre los centinelas resulta mucho menor. Bajo estas condiciones el cluster es, en la práctica, indestructible.

8.3. Topología de comunicación

El subsistema de tolerancia a fallas combina tres planos de comunicación distintos, cada uno con un protocolo elegido según su naturaleza. La siguiente figura los resume.



Los *workers* emiten su *heartbeat* por UDP hacia las N réplicas de centinela. Entre sí, cada par de centinelas intercambia *heartbeats* por UDP de forma continua y mensajes de control Bully por TCP solo durante una elección. El líder repone vía **docker restart** tanto a los *workers* caídos como a los centinelas que dejaron de emitir su *heartbeat*.

Plano	Protocolo	Frecuencia	Propósito
Worker → Centinela	UDP	~5 s (con <i>jitter</i>)	<i>Heartbeat</i> : el <i>worker</i> avisa que está vivo.
Centinela ↔ Centinela	UDP	~1 s	<i>Heartbeat</i> mutuo: detectar la caída del líder y la de cualquier centinela.
Centinela ↔ Centinela	TCP	Solo en elecciones	Control Bully: Election / OK / Coordinator.
Líder → nodo caído	Docker	Ante caída detectada	<code>docker restart</code> del contenedor (<i>worker</i> o centinela).

8.3.1. Trade-offs

La pregunta más interesante de esta sección no es *qué* hace cada plano, sino *por qué* se eligió el protocolo de transporte que usa. La decisión pasa por usar protocolos distintos para necesidades distintas, en lugar de uniformar todo bajo un único mecanismo.

El *heartbeat* de los *workers*. Para este plano se eligió UDP. El *heartbeat* es un mensaje efímero y frecuente del que solo interesa el más reciente, y esas características se ajustan bien a UDP. Al ser un protocolo *fire-and-forget*, no necesita abrir ni mantener una conexión con cada *worker*, de modo que muchos emisores pueden reportar a un mismo centinela sin que este acumule estado de conexiones. La pérdida ocasional de un datagrama tampoco representa un problema, porque un nodo se considera caído recién tras varios *heartbeats* ausentes y no tras uno solo. Por último, el sentido de la comunicación también es el adecuado, ya que es el *worker*, que sabe que está vivo, quien lo informa, y no el centinela el que debe sondear uno por uno a todos los nodos.

Podría ser oportuno reutilizar el *middleware* que ya media el resto de la comunicación pero emplear colas para la señal de salud sería un error. En primer lugar, acoplaría la detección de fallas al propio *broker*, cuando esa señal debe viajar por un canal independiente del plano de datos, porque si el *broker* se degradara perderíamos justamente la capacidad de detectar y reaccionar ante las fallas, que es cuando más la necesitamos. Además, las colas están pensadas para persistir y entregar los mensajes en orden, mientras que un *heartbeat* es desechable y solo interesa el último, de modo que uno viejo acumulado en una cola no aporta información sino ruido. A todo esto se suma que el enrutamiento y la persistencia del *broker* representan un sobre costo innecesario para un mensaje de unos pocos bytes que se envía cada segundo.

La coordinación entre centinelas. El control del algoritmo Bully, en cambio, viaja sobre TCP, porque aquí la prioridad se invierte y se prioriza la correctitud sobre la eficiencia. Como se anticipó, perder un mensaje de respuesta o de proclamación de liderazgo puede derivar en dos líderes simultáneos y, con ellos, en reinicios duplicados o en ningún reinicio, y como el algoritmo presupone un canal confiable, esa precondition descarta a UDP para este plano. TCP aporta exactamente la entrega ordenada y confiable que se necesita, y su costo, que es el *handshake* y el mantenimiento de la conexión, resulta asumible porque estos mensajes son poco frecuentes, se intercambian solo durante una elección y se envían mediante conexiones efímeras que se abren y cierran por mensaje.

Puede llamar la atención que los centinelas usen TCP para el control pero UDP para vigilarse la vida entre ellos, aunque, lejos de ser una inconsistencia, es la misma lógica aplicada de nuevo. Ese *heartbeat* mutuo es apenas un “sigo vivo” de alta frecuencia, aproximadamente uno por segundo, y tolerante a pérdidas, al punto de que hace falta que el *heartbeat* de un centinela falte durante varios periodos seguidos antes de darlo por caído, precisamente para que un datagrama perdido aislado no dispare una elección o reinicio innecesario.

De esta forma, separamos entre un plano de **liveness** (UDP, frecuente y desechable) y un plano de **control** (TCP, ocasional y crítico) porque cada canal emplea el protocolo que su naturaleza

exige, en lugar de forzar un compromiso único que sería subóptimo para ambos.

8.4. Reconexión del cliente

Frente a una falla de conexión entre un cliente y su *gateway*, ambas partes reaccionan de la siguiente manera:

- **Gateway:** Cuando el *gateway* detecta que la conexión se perdió, publica un mensaje *ClientAborted* en el exchange fanout *client_aborts*. Todos los nodos *worker* del sistema están suscritos a este exchange. Al recibir el mensaje, eliminan automáticamente todo el estado asociado a ese cliente, tanto en memoria como en disco, conservando únicamente su *tombstone*.
- **Cliente:** Al detectar la desconexión, el cliente no intenta retomar la sesión anterior. En su lugar, descarta los resultados obtenidos hasta ese momento, selecciona aleatoriamente otro *gateway* y reinicia el procesamiento utilizando un mecanismo de *backoff* exponencial para los reintentos.

Si la desconexión fue consecuencia de la caída del nodo *gateway*, el *sentinel* líder detecta dicha falla y procede a revivirlo. Una vez recuperado, el *gateway* restaura su estado persistido en disco y envía un mensaje *ClientAborted* por cada cliente que estaba atendiendo al momento de la caída.

La decisión de reiniciar el procesamiento desde cero se tomó porque permitir la reanudación de una sesión requeriría que el *gateway* almacenara de forma durable, para cada cliente, qué batches recibió y qué resultados ya entregó. Además, obligaría al cliente a reconectarse siempre al mismo *gateway* o, alternativamente, a compartir ese progreso entre todos los *gateways*. Esto introduciría complejidad y sobrecarga adicional en el *gateway*, que constituye el principal cuello de botella del sistema. Reiniciar desde cero resulta una solución más simple y robusta, ya que cualquier *gateway* puede hacerse cargo del cliente sin necesidad de recuperar contexto previo.

8.5. Persistencia y recuperación del estado

Revivir un proceso caído es una condición necesaria pero no suficiente. Cuando un *worker* muere a mitad del procesamiento de un cliente, al reiniciarse debe recuperar el estado que había acumulado y, además, evitar volver a procesar los mensajes que ya había procesado antes de morir. De lo contrario, un reinicio introduciría resultados incompletos o duplicados. Esta recuperación se apoya en la **persistencia** del estado a disco y la **deduplicación** de los mensajes reentregados.

El punto de partida es una propiedad del *middleware*: con confirmaciones manuales, el *broker* garantiza una semántica de entrega *at-least-once*. Si un *worker* desencola un mensaje y muere **antes** de confirmarlo, el *broker* se lo reentrega cuando vuelve a estar disponible. Esto implica que un mismo mensaje puede llegar a procesarse dos veces. Todo el diseño de persistencia y deduplicación gira en torno a neutralizar ese posible duplicado, de modo que el efecto observable sea el de un procesamiento único.

8.5.1. ¿Cuándo se persiste?

Un *worker* no persiste su estado ni confirma cada mensaje de forma individual, sino que agrupa el trabajo en **puntos de control**, acumulando en memoria los resultados de varios batches para volcarlos y confirmarlos en bloque. Un punto de control se dispara por cualquiera de tres motivos. El más común es haber acumulado una cantidad fija de batches sin confirmar. El segundo es el paso del tiempo, ya que un umbral de unos diez segundos desde el primer mensaje pendiente fuerza el volcado aunque no se haya alcanzado ese volumen, de forma que los datos de un cliente no queden indefinidamente sin persistir. El tercero es un tope a la acumulación en memoria. Mientras los batches no se confirman, sus resultados quedan retenidos en el *worker*, y el *broker* solo entrega una cantidad limitada de mensajes sin confirmar a la vez. Cuando esa cantidad, sumada sobre todos los clientes, alcanza ese límite, el *worker* fuerza un volcado para confirmar lo pendiente,

liberar memoria y poder seguir recibiendo. Agrupar el trabajo de esta forma amortiza el costo de la escritura a disco y de las confirmaciones sobre muchos mensajes, en lugar de pagarlo en cada uno.

El orden interno de un punto de control es la invariante más importante del diseño, y consiste en **persistir antes de confirmar**. La secuencia es estricta:

1. Publicar al *broker* los resultados acumulados hacia los nodos siguientes.
2. Esperar las **confirmaciones de publicación** del *broker*, que aseguran que esos resultados ya están en las colas aguas abajo.
3. Escribir el estado propio a disco de forma durable.
4. **Recién entonces**, confirmar los mensajes de entrada al *broker*.

La confirmación de un mensaje es una promesa de que “esto ya está a salvo, no me lo reenvíes”. Si esa promesa se hiciera antes de que el estado fuera durable, una muerte en el medio dejaría un mensaje confirmado, que el *broker* no reenviará y, por ende, una pérdida silenciosa de datos. Para reforzar la invariante, ante cualquier error en este camino crítico el *worker* aborta de inmediato (*fail-fast*) sin confirmar. Se prefiere morir y dejar que el *broker* reentregue, dado que reprocesar es seguro gracias a la deduplicación, antes que arriesgar una inconsistencia.

8.5.2. ¿Cómo se persiste?

La primera implementación de la persistencia escribía, en cada punto de control, una **snapshot completa** del estado del nodo. Se serializaba todo el estado y se lo grababa a un archivo de forma atómica (escritura a un archivo temporal, volcado a disco y renombrado atómico, que es el punto de compromiso). Este esquema es correcto y simple, pero tiene un problema de **rendimiento** para los nodos cuyo estado *crece con los datos* (por ejemplo, los que acumulan un grafo de cuentas o un diccionario de agregación).

El problema es que reescribir el estado completo en cada punto de control vuelve el costo total de escritura **cuadrático**. Si el estado crece hasta un tamaño N , a lo largo de la corrida se realizan del orden de N puntos de control y en cada uno se reescriben los bytes, cada vez más numerosos, del estado acumulado.

La solución fue cambiar a un **registro de escritura anticipada** (*Write-Ahead Log*, WAL). En lugar de reescribir todo el estado, en cada punto de control se **anexa un delta compacto**: solamente el cambio que ese grupo de batchs produjo sobre el estado (típicamente, las pocas entradas nuevas de un mapa o de un conjunto). El estado completo se vuelca a una *snapshot* solo **de forma esporádica**. Así, el total de bytes escritos pasa de $O(N^2)$ a $O(N)$.

Ahora bien, este cambio se aplica de forma selectiva, solo donde realmente aporta. El registro de escritura anticipada se reserva para los nodos que acumulan estado grande, es decir, aquellos cuyo estado crece con los datos a lo largo de la corrida, como el agregador por banco, el filtro de cuentas sospechosas o el reconstructor de caminos. El resto de los nodos, como los filtros, los *sharders* y los *joiners*, maneja un estado pequeño o acotado y conserva el esquema original de *snapshot* completa, que para ellos es menos denso y no justifica la complejidad adicional del log.

El registro de escritura anticipada se apoya en tres ideas. La primera es que cada delta se anexa como un registro auto-delimitado que incluye su longitud y un *checksum*. De ese modo, si el proceso muere a mitad de una escritura, el último registro queda truncado o con la suma inválida, y en la recuperación tanto ese registro como todo lo que le siga se descartan. Esto es correcto, porque una escritura a medias implica que la confirmación nunca llegó a ocurrir y, por lo tanto, ese batch será reentregado de todas formas.

El registro de escritura anticipada se apoya en tres ideas. La primera es que cada delta se anexa como un registro auto-delimitado que incluye su longitud y una suma de verificación. De ese modo, si el proceso muere a mitad de una escritura, el último registro queda truncado o con la suma

inválida, y en la recuperación tanto ese registro como todo lo que le siga se descartan. Esto es correcto, porque una escritura a medias implica que la confirmación nunca llegó a ocurrir y, por lo tanto, ese batch será reentregado de todas formas.

La segunda idea es que cada *snapshot* completa lleva una etiqueta que marca hasta qué punto del log ya quedó incorporado en ella. Dicho de otro modo, la *snapshot* refleja el efecto de todos los registros anteriores a esa marca, y los registros posteriores son los únicos que todavía no están incluidos. Gracias a esto, la recuperación es siempre la misma operación, que consiste en cargar la última *snapshot* y reaplicar solo los registros del log que vienen después de esa etiqueta. Esa única pasada reconstruye tanto el estado de negocio como la información de deduplicación, y por eso esta última viaja dentro del log y no en un archivo aparte.

La tercera idea es la compactación. Para que el log no crezca sin límite, y para que la recuperación no deba reproducir demasiados registros, cada tanto se vuelca una *snapshot* nueva y se comienza un log vacío. Esto ocurre cuando el log supera cierto múltiplo del tamaño de la última *snapshot*, y en todo caso al cerrar el cliente. La rotación resulta segura ante una caída en cualquier instante, ya que la etiqueta de la *snapshot* desambigua cuál es el log vigente y los restos de una compactación interrumpida se ignoran y se limpian al reabrir.

Cabe notar, por último, que el registro de escritura anticipada no reduce la frecuencia de escritura a disco, ya que se sigue persistiendo una vez por punto de control con la misma cadencia, sino el tamaño de cada escritura, que pasa de megabytes, cuando se vuelca el estado entero, a apenas los bytes del delta. El cuello de botella medido eran los *bytes* escritos, no la cantidad de operaciones de disco.

8.5.3. Deduplicación

Como el *broker* puede reentregar un mensaje, cada *worker* debe reconocer los que ya procesó y descartarlos. Para lograrlo, cada mensaje viaja con un **encabezado** que lo identifica de forma unívoca. Los datos de ese encabezado que importan para la deduplicación son los siguientes:

- **Identificador de cliente.** A qué cliente y corrida pertenece el mensaje. Es la base de toda clave de deduplicación.
- **Número de secuencia.** Un contador que numera el mensaje en su origen y se acarrea sin cambios por las etapas que solo lo reenvían.
- **Emisor original.** La etapa y la réplica que generaron esa numeración.
- **Emisor inmediato.** La etapa y la réplica que pusieron el mensaje en la cola en este salto, que no siempre coincide con el emisor original.
- **Camino de ramificación.** La salida que tomó el mensaje en cada bifurcación, como la que separa las tuplas que cumplen un predicado de las que no.

Con estos campos, el *worker* arma la **clave de deduplicación** de cada mensaje combinando el identificador de cliente con un *espacio de identificación* formado por el emisor original, el emisor inmediato y el camino de ramificación. Para los datos, el nodo recuerda qué números de secuencia ya vio dentro de cada espacio. Si un número ya figura como visto, descarta el mensaje y lo confirma, y si es nuevo, lo procesa y lo registra. Ese registro se persiste junto con el estado, así que al revivir el nodo recuerda qué había procesado. Para los mensajes de control, como el EOF o los *tokens* del anillo, alcanza con guardar el último número visto por emisor y descartar los que tengan un número menor o igual.

Dos detalles explican la forma de esta clave. Por un lado, incluir tanto al emisor original como al inmediato evita falsas colisiones, ya que un nodo que une flujos de varias fuentes recibe numeraciones que arrancan desde uno y se solapan, y distinguir quién generó el número y quién lo reenvió mantiene esos espacios separados. Por otro lado, en lugar de recordar solo el mayor número visto, el nodo guarda el conjunto de números vistos, porque puede entregar los mensajes fuera de

orden y una simple marca de agua descartaría sin razón a los atrasados. Para que ese conjunto no crezca con la cantidad de mensajes, se lo comprime como un conjunto de intervalos, de modo que un flujo ordenado se reduce a un único rango de unos pocos bytes.

Por último, la deduplicación es local a cada nodo: cada réplica mantiene su registro de lo visto en memoria y lo reconstruye desde el disco al revivir. Para que funcione, un mensaje reentregado debe volver siempre al mismo nodo, y eso se logra dándole una cola propia a cada partición de cada etapa y enrutando cada batch a una sola partición con un *hash* determinístico de una clave. Esa clave de ruteo varía según la etapa (el cliente, el cliente más el número de secuencia, o la entidad que se agrupa), pero al ser determinística el mismo batch cae siempre en la misma partición, y por lo tanto en el mismo nodo. No hay que confundir esta clave de ruteo, que decide a qué nodo llega el batch, con la clave de deduplicación, que el nodo usa para reconocer si ya lo procesó. Además, como las colas tienen nombre fijo y no se eliminan al quedarse sin consumidores, un mensaje dirigido a un nodo caído queda encolado esperándolo, y como el consumo se confirma de forma manual, si el nodo muere antes de confirmar el *broker* se lo reentrega al reconectarse.

Queda un caso al borde del ciclo de vida de un cliente. Cuando un cliente termina, ya sea porque completó su procesamiento o porque fue abortado tras una reconexión, el nodo libera su registro de números de secuencia para no acumular el estado de clientes que ya no volverán. Pero un mensaje suyo todavía en vuelo podría llegar después de esa limpieza y, al no encontrar registro, ser tratado como nuevo y recrear el estado del cliente. Para evitarlo, el nodo deja una **lápida** (*tombstone*): una marca liviana que recuerda, con su fecha, que ese cliente ya terminó. Cualquier mensaje de un cliente con lápida se descarta sin procesar. La lápida se persiste junto con el resto del estado y se borra recién tras un tiempo de vida holgado, suficiente para que todos los mensajes rezagados ya hayan drenado.

Con todo esto, logramos que el sistema sea tolerante a fallos. Cuando el Chaos Monkey mata un *worker* y los centinelas lo reinician, el mensaje que quedó sin confirmar se reentrega y vuelve al mismo nodo, que ya reconstruyó su estado desde el disco, lo reconoce como duplicado y lo descarta. El resultado es un procesamiento **exactly-once**, sin pérdida de datos, porque se persiste antes de confirmar, y sin duplicados, porque la deduplicación absorbe la reentrega.

9. Planificación y Organización

En la presente sección consignamos el listado de tareas que componen el desarrollo del sistema, junto con la división tentativa entre los integrantes del equipo. La granularidad de las tareas se eligió de modo de coincidir, en líneas generales, con los componentes identificados en las vistas anteriores, lo que facilitará la integración y reducirá las dependencias entre integrantes durante la implementación.

9.1. Listado de Tareas a Ejecutar

- **Cliente.** Implementación del lector de CSV, armado y envío de *batches* por TCP, manejo de la señal de EOF hacia el *gateway* y recepción de los resultados consolidados.
- **Gateway + Tagger.** Implementación del servidor TCP, asociación de **ClientID** a sockets, lógica de *tagging* por contenido de transacción, contadores por *topic* para inyectar el total esperado en los EOFs y reenvío de los resultados consolidados al cliente correspondiente.
- **Protocolo cliente ↔ gateway.** Definición del formato de *batches*, señal de EOF y mensajes de respuesta. Implementación dentro del paquete transversal **generic.communication**.
- **Protocolo de anillo de EOF.** Implementación de las dos rondas (colecta y envío) descritas en la sección *Manejo de EOF*, así como el mecanismo de reintento ante discrepancia con el total esperado.
- **Workers de Q1.** Implementación del *Dispatcher* con anillo de control y emisión a **Results**.

- **Workers de Q2.** Implementación del *Max* local con anillo de control y del *Aggregator* con *sharding* por cliente.
- **Workers de Q3.** Implementación de las dos sub-ramas: *Sum* con anillo de control, *Aggregator* y *Joiner* para el *Average Calculator*; y *Period filter* con persistencia local de transacciones a la espera del promedio.
- **Workers de Q4.** Implementación del *Sharder*, del *3-Path Finder* (con su estructura interna de cuentas *Entrada/Salida*) y del *Intermediate Account Counter* con *sharding* por par (origen, destino).
- **Workers de Q5.** Implementación del cliente HTTP hacia la API de Frankfurter (paquete `external_apis`), del *Converter* con anillo de control y del *Aggregator* con *sharding* por cliente.
- **Final Joiner.** Implementación del consumidor del *exchange Results* con *sharding* por cliente, gestión del estado parcial por cliente y emisión de la respuesta consolidada a **Final** cuando se reciben los EOFs de las cinco ramas.
- **Despliegue.** Construcción de las imágenes Docker de cada unidad (**client**, **gateway**, **worker**), *compose* de la topología completa con RabbitMQ y configuración de las réplicas iniciales por grupo.
- **Pruebas funcionales.** Verificación de los resultados de cada consulta contra el *notebook* patrón provisto por la cátedra.

9.2. División del Trabajo entre Integrantes

Tarea	Encargado
Cliente	Juan Martín de la Cruz
Gateway + Tagger	Ian Klaus von der Heyde
Protocolo cliente ↔ gateway	Ian Klaus von der Heyde
Protocolo de anillo de EOF	Ian Klaus von der Heyde
Workers de Q1 (Micropagos)	Juan Martín de la Cruz
Workers de Q2 (Máximos)	Juan Martín de la Cruz
Workers de Q3 (Promedios)	Agustín Altamirano
Workers de Q4 (Scatter-Gather)	Agustín Altamirano
Workers de Q5 (Cotizaciones API)	Ian Klaus von der Heyde
Final Joiner	Juan Martín de la Cruz
Despliegue (Docker/Compose)	Agustín Altamirano, Ian Klaus von der Heyde, Juan Martín de la Cruz
Pruebas funcionales	Ian Klaus von der Heyde